



DML

- **DML(Data Manipulation Language)**

데이터 조회 및 변경하기 위한 명령어

- **DML의 종류**

1. 조회 : SELECT
2. 삽입 : INSERT
3. 삭제 : DELETE
4. 변경 : UPDATE

- **스키마 오브젝트(Schema Object)**

테이블, 뷰, 인덱스와 같은 구조를 포함하는 데이터의 저장 구조를 가진 객체

- **스키마(Schema)**

여러 개의 스키마 오브젝트가 모인 집합

사용자가 생성한 데이터베이스 객체들을 의미하며 스키마의 이름은 계정명을 따라감
ex) jaeseop 이라는 계정이 테이블 등의 객체를 만들었다면 jaeseop 스키마가 됨

DML (SELECT-1)

- **SELECT**

데이터를 조회할 때 사용하는 명령어
→ 문법 : SELECT [컬럼명] FROM [테이블명];

- **테이블**

데이터베이스에서 정보를 구조화하고 저장하기 위한 개념

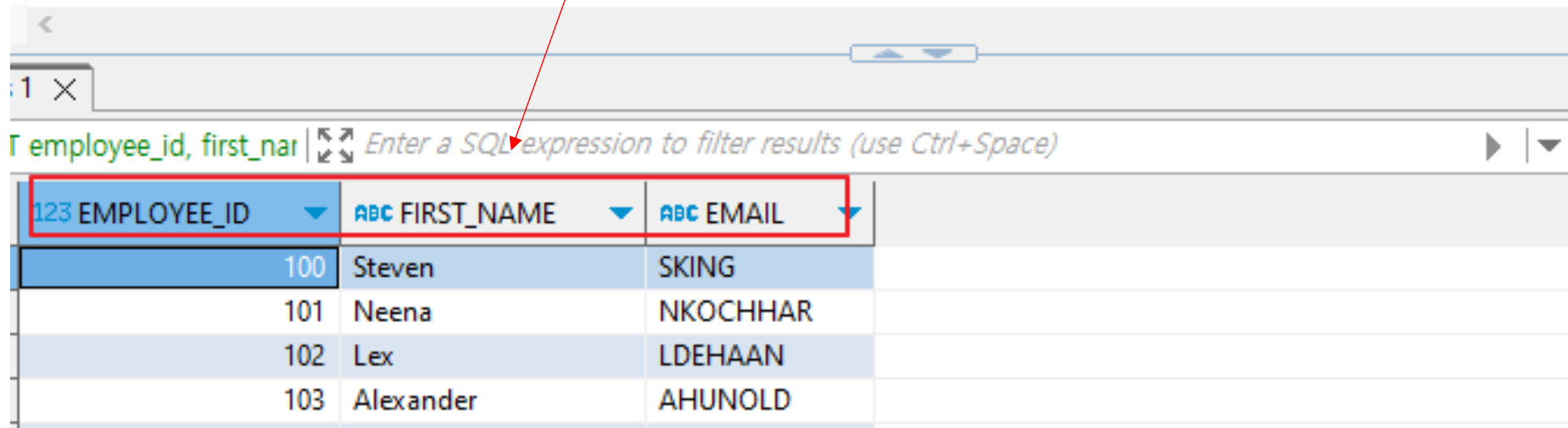
스프레드시트의 시트와 비슷한 개념으로써, 오라클에서 각각의 테이블은 하나의 "표" 형식으로 되어있음

DML

• SELECT

각각의 컬럼을 조회할 수 있으며, SELECT를 통해 조회된 데이터의 결과물을 RESULT SET 이라고 함

```
SELECT employee_id, first_name, email FROM EMPLOYEES;
```



123 EMPLOYEE_ID	ABC FIRST_NAME	ABC EMAIL
100	Steven	SKING
101	Neena	NKOCHHAR
102	Lex	LDEHAAN
103	Alexander	AHUNOLD

• 컬럼

컬럼은 테이블 내의 각 열을 정의 함

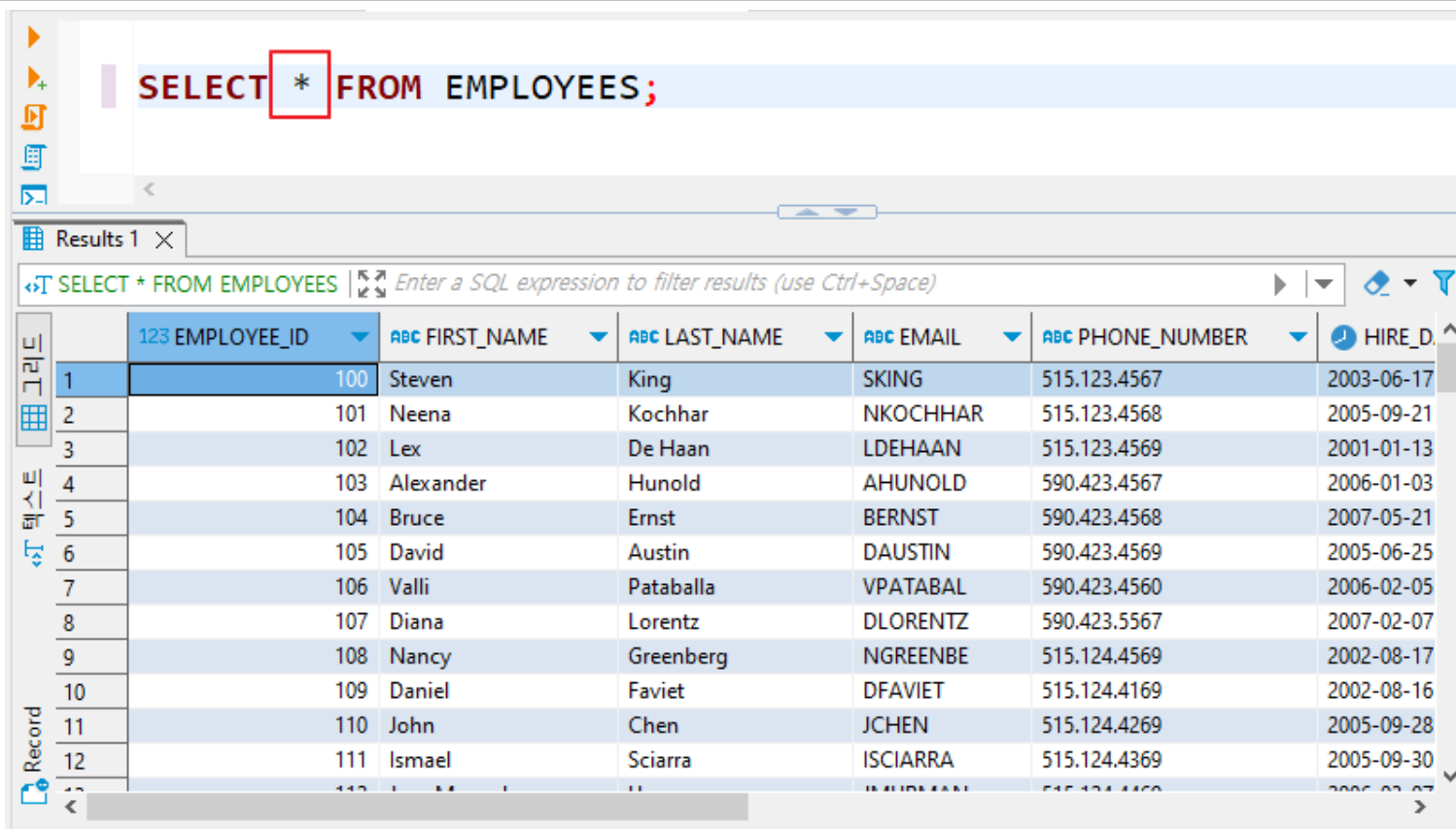
* HTML table 태그의 th와 같음

컬럼

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
100	Steven	King	SKING	515.123.4567	2003-06-17 00:00:00.000	AD_PRES
101	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21 00:00:00.000	AD_VP
102	Lex	De Haan	LDEHAAN	515.123.4569	2001-01-13 00:00:00.000	AD_VP
103	Alexander	Hunold	AHUNOLD	590.423.4567	2006-01-03 00:00:00.000	IT_PROG
104	Bruce	Ernst	BERNST	590.423.4568	2007-05-21 00:00:00.000	IT_PROG
105	David	Austin	DAUSTIN	590.423.4569	2005-06-25 00:00:00.000	IT_PROG
106	Valli	Pataballa	VPATABAL	590.423.4560	2006-02-05 00:00:00.000	IT_PROG
107	Diana	Lorentz	DLORENTZ	590.423.5567	2007-02-07 00:00:00.000	IT_PROG

• SELECT

컬럼명을 작성하는 공간에 *를 입력하면, 모든 컬럼에 대한 정보를 조회함
* 단, 불필요한 컬럼도 조회를 하기 때문에 성능상의 이유로 *는 되도록 사용하지 않는것을 권장함



The screenshot shows a SQL IDE interface. At the top, a query editor contains the text `SELECT * FROM EMPLOYEES;`. The asterisk (*) is highlighted with a red square. Below the editor, a results pane titled "Results 1" displays the output of the query. The results are shown in a table with columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, and HIRE_DATE. The first 12 rows are visible, starting with Steven King (ID 100) and ending with Ismael Sciarra (ID 111).

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE
1	100	Steven	King	SKING	515.123.4567	2003-06-17
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21
3	102	Lex	De Haan	LDEHAAN	515.123.4569	2001-01-13
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	2006-01-03
5	104	Bruce	Ernst	BERNST	590.423.4568	2007-05-21
6	105	David	Austin	DAUSTIN	590.423.4569	2005-06-25
7	106	Valli	Pataballa	VPATABAL	590.423.4560	2006-02-05
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	2007-02-07
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	2002-08-17
10	109	Daniel	Faviet	DFAVIET	515.124.4169	2002-08-16
11	110	John	Chen	JCHEN	515.124.4269	2005-09-28
12	111	Ismael	Sciarra	ISCIARRA	515.124.4369	2005-09-30

• Q1. 실습

1. EMPLOYEES 테이블의 EMPLOYEE_ID, LAST_NAME, EMAIL 컬럼 조회
2. EMPLOYEES 테이블의 JOB_ID 컬럼 조회
3. JOBS 테이블의 JOB-TITLE, MIN_SALASRY, MAX_SALARY 컬럼 조회
4. DEPARTMENTS 테이블의 모든 컬럼 조회

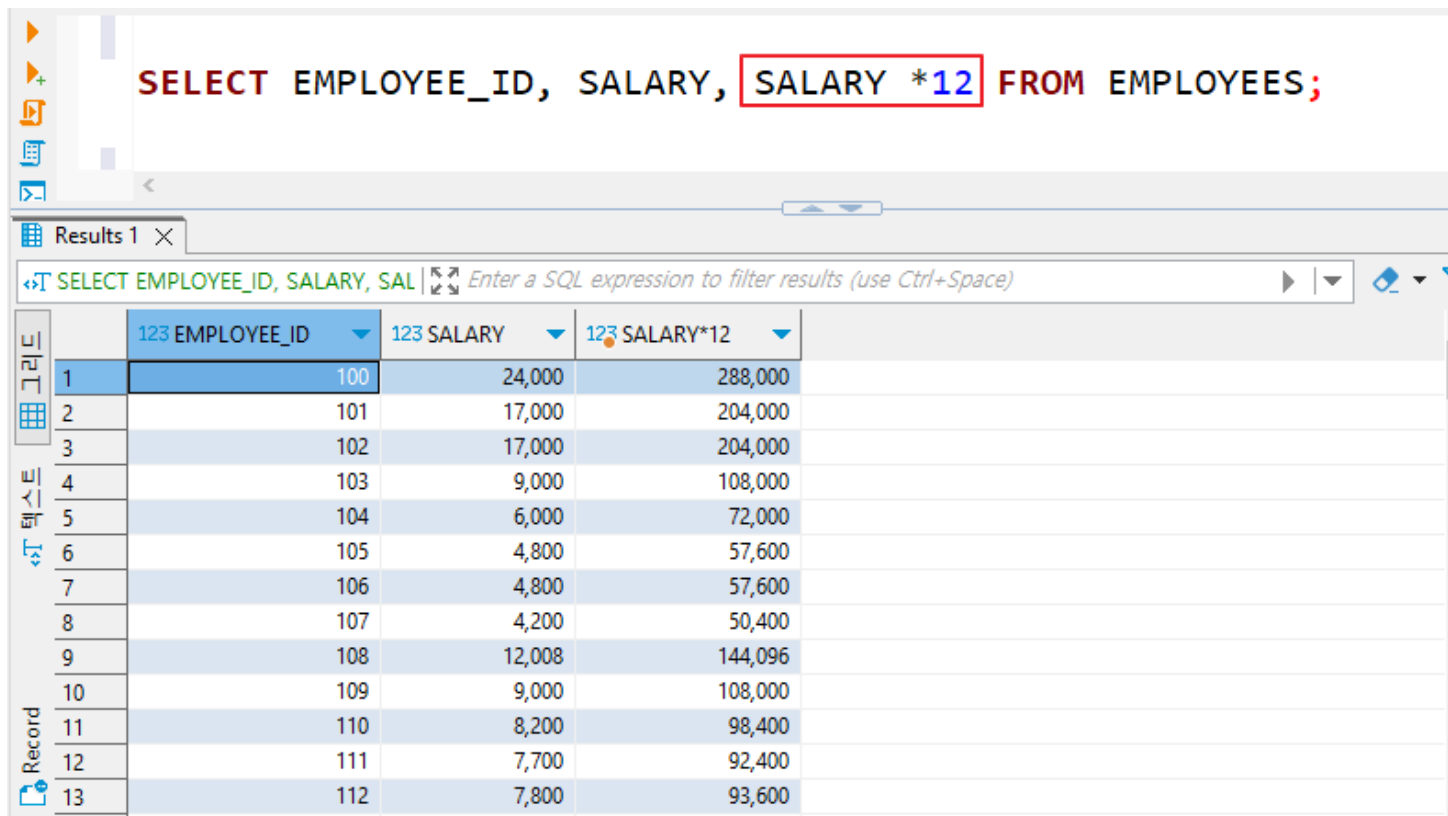
DML (연산자)

• 연산자 우선순위

1. ()
2. 산술 연산자 (*, /, +, -)
3. 연결 연산자 (||)
4. 비교 연산자 (=, >, <, >=, <=, <>, !=, ^=)
5. IS NULL, NULL, LIKE, IN
6. BETWEEN AND
7. NOT (논리 부정 연산자)
8. AND
9. OR

- 산술 연산자

DB에서도 산술 연산자를 활용할 수 있는데, 사칙 연산(+, -, *, /)이 있음



The screenshot shows a SQL query editor with the following query:

```
SELECT EMPLOYEE_ID, SALARY, SALARY * 12 FROM EMPLOYEES;
```

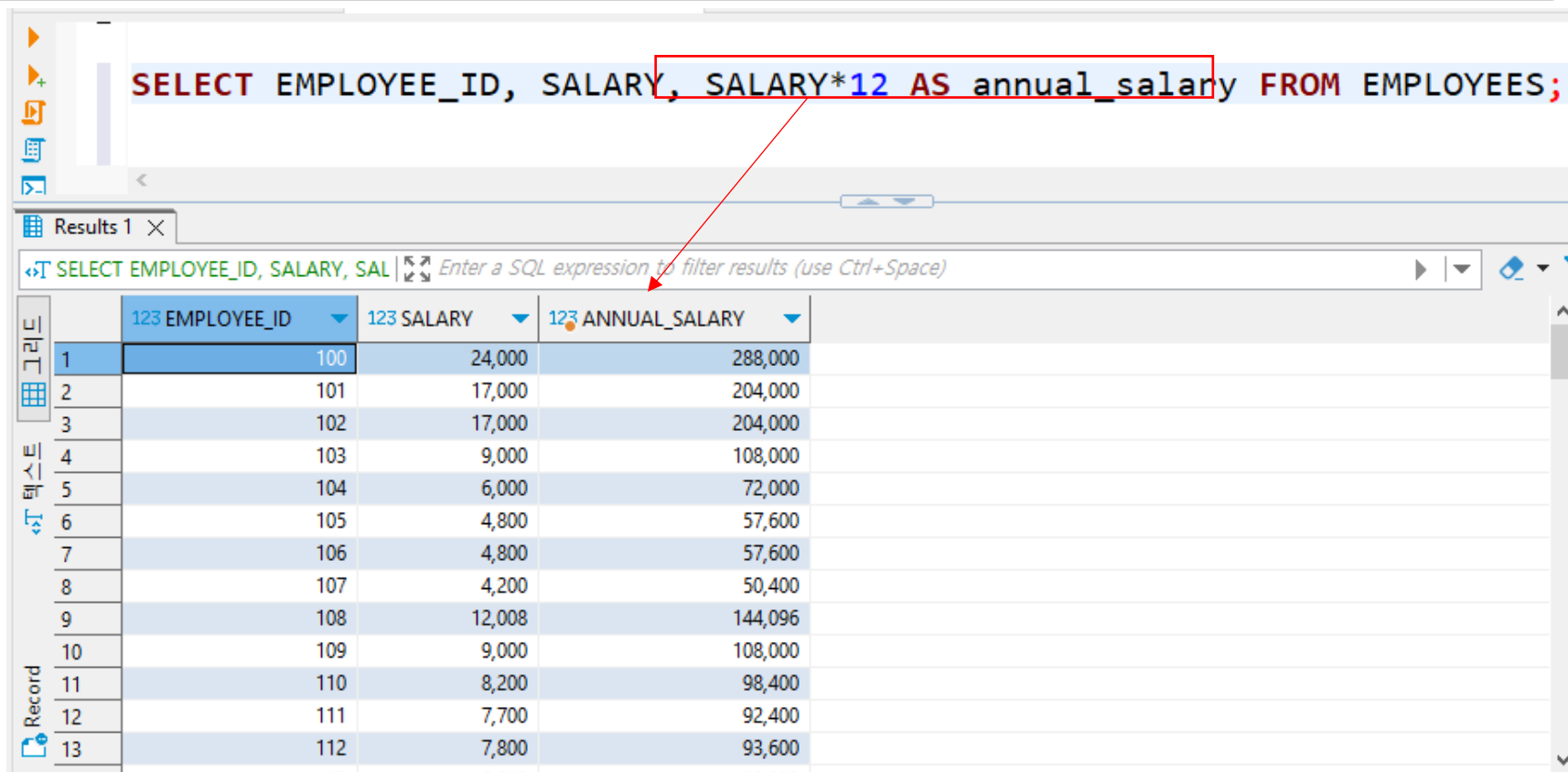
The results are displayed in a table grid with 13 rows and 4 columns. The columns are labeled: 123 EMPLOYEE_ID, 123 SALARY, and 123 SALARY*12. The first row is highlighted in blue.

	123 EMPLOYEE_ID	123 SALARY	123 SALARY*12
1	100	24,000	288,000
2	101	17,000	204,000
3	102	17,000	204,000
4	103	9,000	108,000
5	104	6,000	72,000
6	105	4,800	57,600
7	106	4,800	57,600
8	107	4,200	50,400
9	108	12,008	144,096
10	109	9,000	108,000
11	110	8,200	98,400
12	111	7,700	92,400
13	112	7,800	93,600

• 별칭

AS를 활용하여 컬럼명에 별칭을 부여할 수 있음

* AS 생략 가능



The screenshot shows a SQL query editor with the following query:

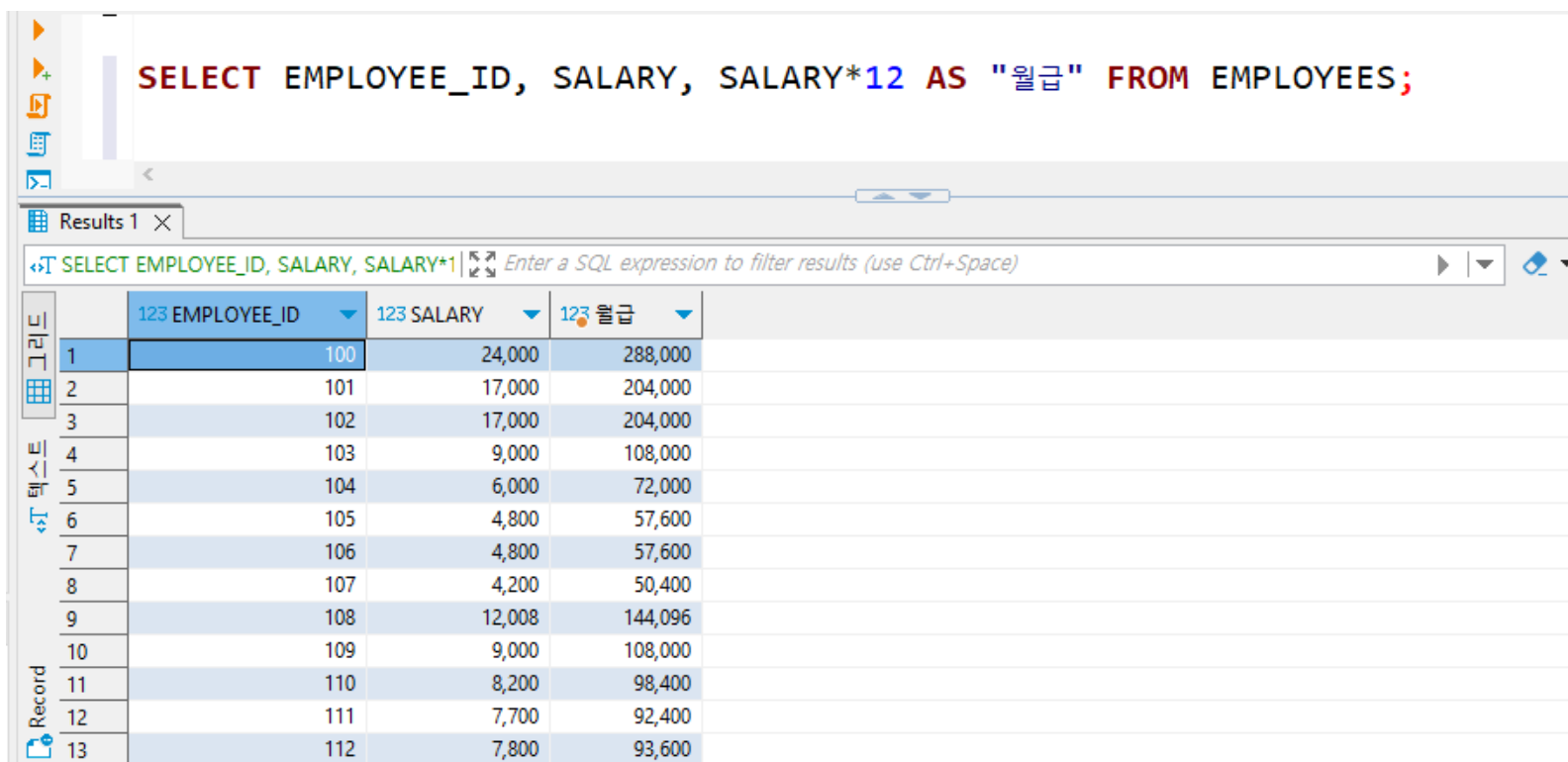
```
SELECT EMPLOYEE_ID, SALARY, SALARY*12 AS annual_salary FROM EMPLOYEES;
```

The query is executed, and the results are displayed in a table. The table has three columns: EMPLOYEE_ID, SALARY, and ANNUAL_SALARY. The ANNUAL_SALARY column is calculated as SALARY * 12. The results are as follows:

	EMPLOYEE_ID	SALARY	ANNUAL_SALARY
1	100	24,000	288,000
2	101	17,000	204,000
3	102	17,000	204,000
4	103	9,000	108,000
5	104	6,000	72,000
6	105	4,800	57,600
7	106	4,800	57,600
8	107	4,200	50,400
9	108	12,008	144,096
10	109	9,000	108,000
11	110	8,200	98,400
12	111	7,700	92,400
13	112	7,800	93,600

- 별칭

한글도 사용 가능함

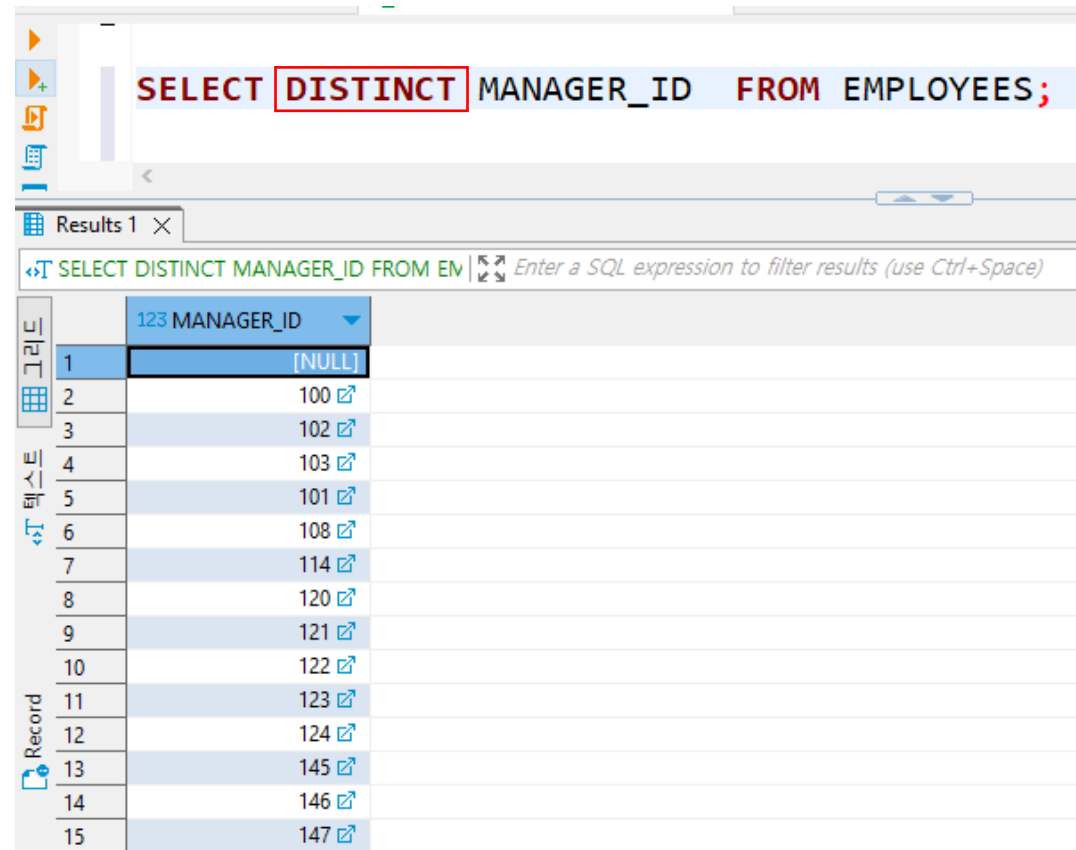


The screenshot shows a SQL IDE interface. At the top, a query is entered: `SELECT EMPLOYEE_ID, SALARY, SALARY*12 AS "월급" FROM EMPLOYEES;`. Below the query editor, the results are displayed in a table. The table has four columns: `EMPLOYEE_ID`, `SALARY`, and `월급` (which is `SALARY*12`). The results are numbered 1 through 13. The first row shows `EMPLOYEE_ID` 100, `SALARY` 24,000, and `월급` 288,000. The subsequent rows show `EMPLOYEE_ID` 101 through 112 with their respective salaries and calculated monthly amounts.

	123 EMPLOYEE_ID	123 SALARY	123 월급
1	100	24,000	288,000
2	101	17,000	204,000
3	102	17,000	204,000
4	103	9,000	108,000
5	104	6,000	72,000
6	105	4,800	57,600
7	106	4,800	57,600
8	107	4,200	50,400
9	108	12,008	144,096
10	109	9,000	108,000
11	110	8,200	98,400
12	111	7,700	92,400
13	112	7,800	93,600

- **DISTINCT**

조회할 컬럼의 데이터에서 중복을 제거하고 조회



The screenshot shows a SQL query editor with the following query:

```
SELECT DISTINCT MANAGER_ID FROM EMPLOYEES;
```

The results are displayed in a grid view with the following data:

123 MANAGER_ID
[NULL]
100
102
103
101
108
114
120
121
122
123
124
145
146
147

• Q2. 실습

1. EMPLOYEES 테이블에서 부서(DEPARTMENTS) 아이디 조회
→ 중복을 제거하여 조회하세요.
2. EMPLOYEES 테이블에서 월급(SALARY), 연봉, 연봉(보너스 포함) 조회
→ 산술 연산자를 사용하세요.
→ 보너스 컬럼은 COMMISSION_PCT 입니다.
→ 연봉과 연봉(보너스포함)은 컬럼명에 별칭을 주세요.

- WHERE

데이터를 조회할 때 특정 조건을 주어 조건에 만족하는 데이터만 조회함

문법 : SELECT [컬럼명] FROM [테이블명] WHERE 조건식;

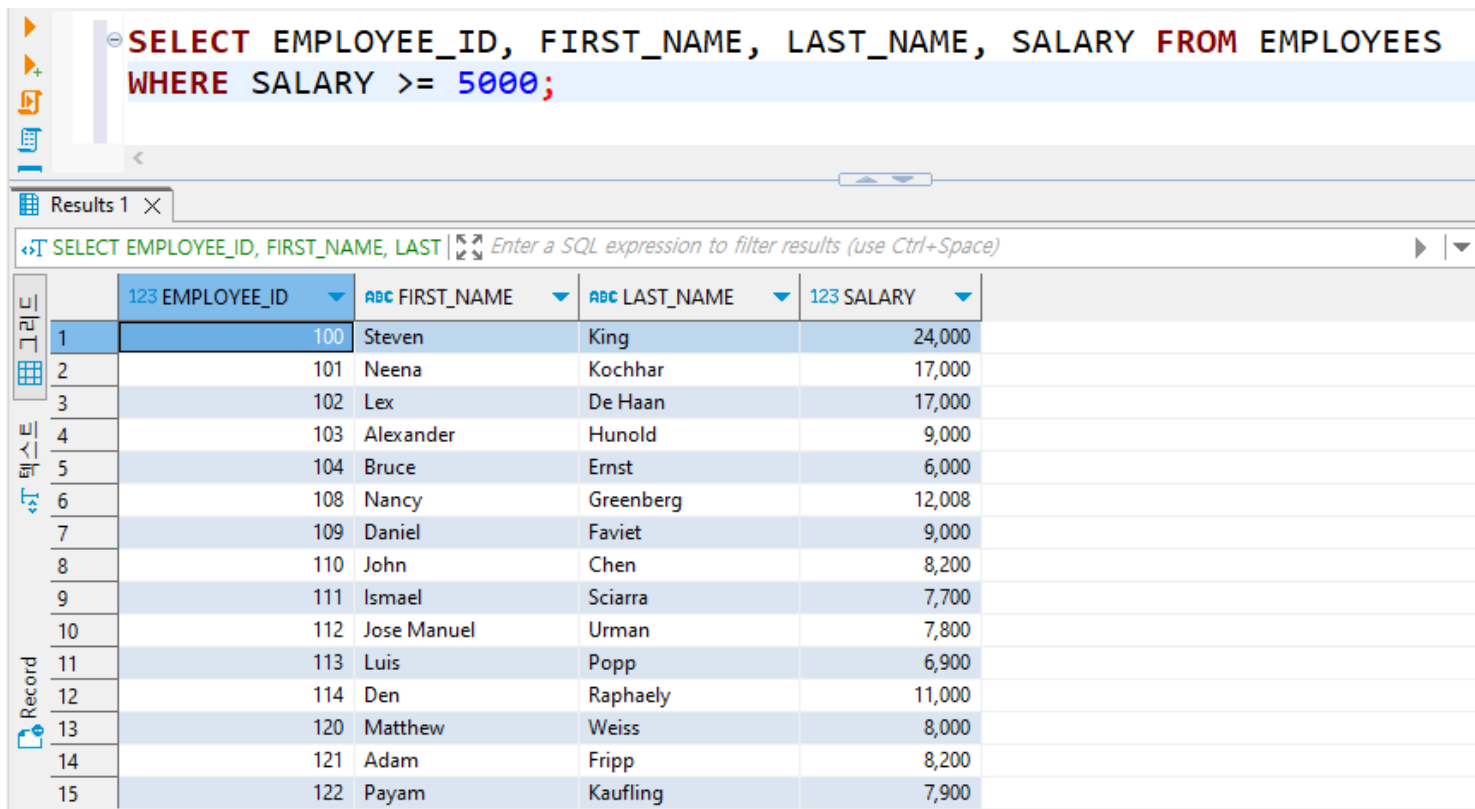
- 비교 연산자

<, <=, >, >=, =, !=

* 다른 비교 연산자는 이전에 배웠던 것과 똑같으며 =는 같다 !=는 같지 않다

- WHERE

EMPLOYEES 테이블에서 월급이 5000 이상인 직원들의 정보 조회

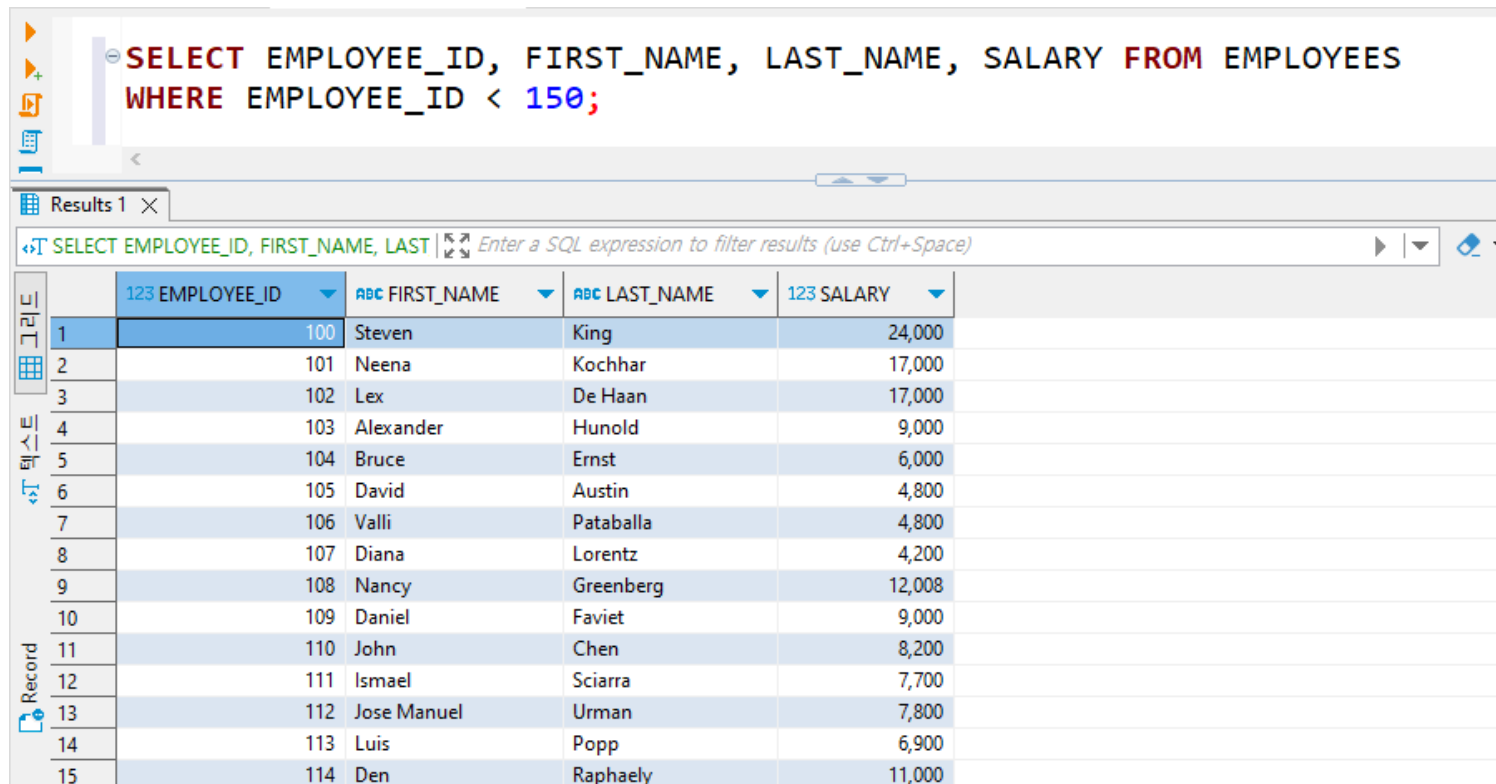


```
SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES
WHERE SALARY >= 5000;
```

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	100	Steven	King	24,000
2	101	Neena	Kochhar	17,000
3	102	Lex	De Haan	17,000
4	103	Alexander	Hunold	9,000
5	104	Bruce	Ernst	6,000
6	108	Nancy	Greenberg	12,008
7	109	Daniel	Faviet	9,000
8	110	John	Chen	8,200
9	111	Ismael	Sciarra	7,700
10	112	Jose Manuel	Urman	7,800
11	113	Luis	Popp	6,900
12	114	Den	Raphaely	11,000
13	120	Matthew	Weiss	8,000
14	121	Adam	Fripp	8,200
15	122	Payam	Kaufling	7,900

- WHERE

직원들의 번호가 150 미만인 직원들만 조회



The screenshot shows a SQL query editor with the following query:

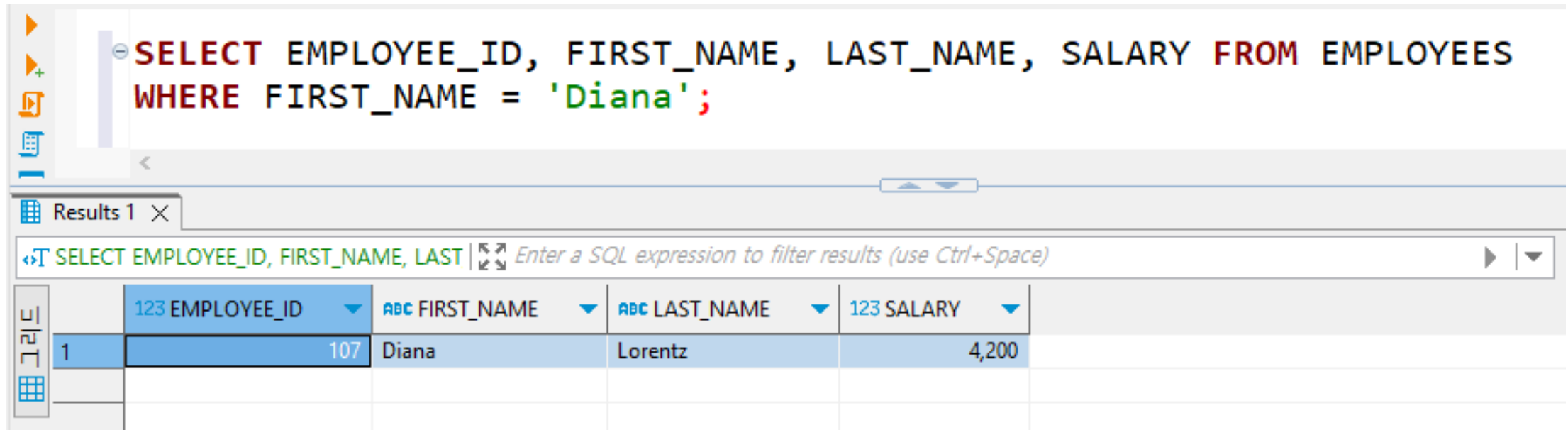
```
SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES  
WHERE EMPLOYEE_ID < 150;
```

Below the query editor, the results are displayed in a table. The table has 5 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, and SALARY. The results are sorted by EMPLOYEE_ID in ascending order. The first 15 rows are visible, showing employees with IDs from 100 to 114.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	100	Steven	King	24,000
2	101	Neena	Kochhar	17,000
3	102	Lex	De Haan	17,000
4	103	Alexander	Hunold	9,000
5	104	Bruce	Ernst	6,000
6	105	David	Austin	4,800
7	106	Valli	Pataballa	4,800
8	107	Diana	Lorentz	4,200
9	108	Nancy	Greenberg	12,008
10	109	Daniel	Faviet	9,000
11	110	John	Chen	8,200
12	111	Ismael	Sciarra	7,700
13	112	Jose Manuel	Urman	7,800
14	113	Luis	Popp	6,900
15	114	Den	Raphaely	11,000

- WHERE

이름이 Diana인 직원 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

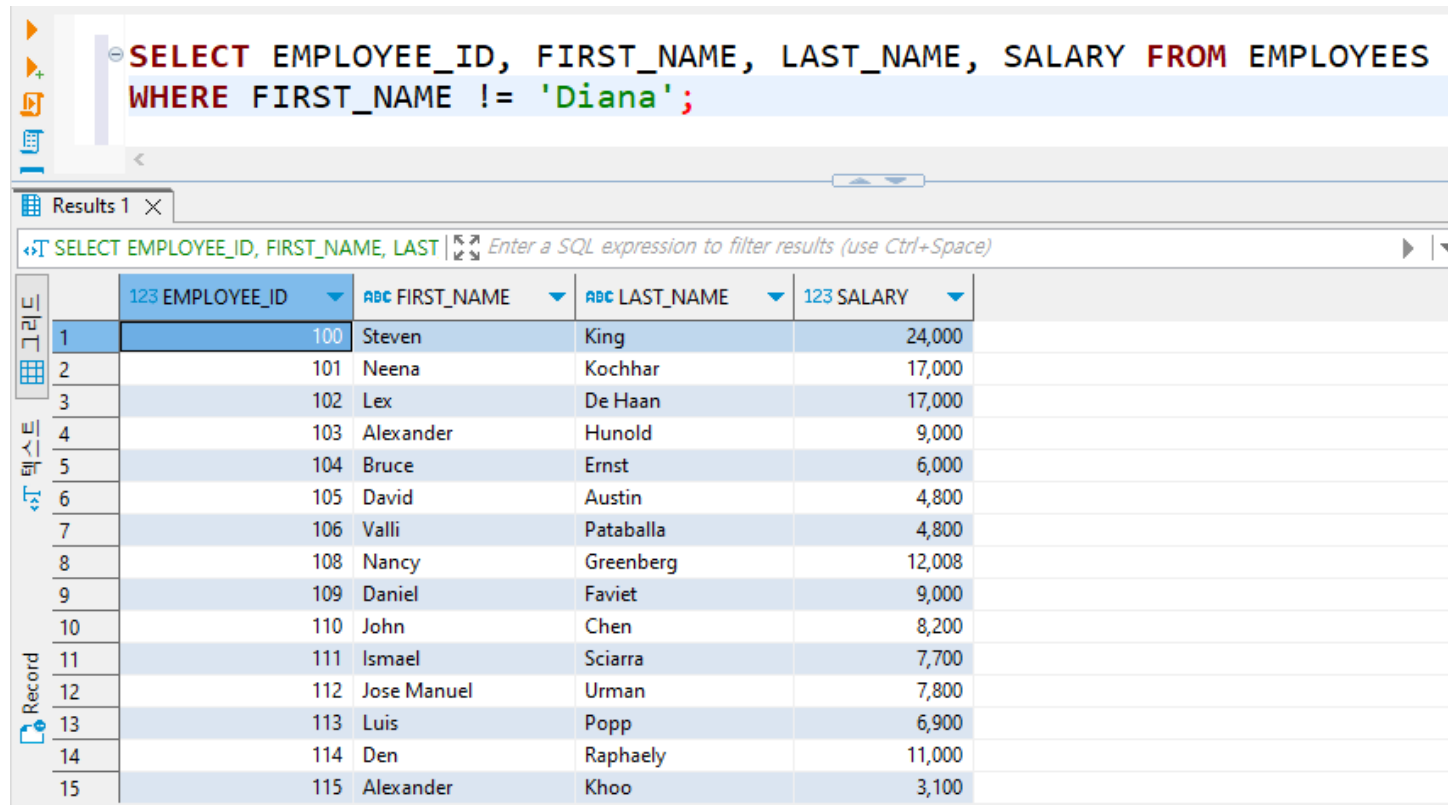
```
SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES  
WHERE FIRST_NAME = 'Diana';
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has five columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, and SALARY. The first row of data shows an employee with ID 107, first name Diana, last name Lorentz, and a salary of 4,200.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	107	Diana	Lorentz	4,200

- WHERE

이름이 Diana가 아닌 직원 조회

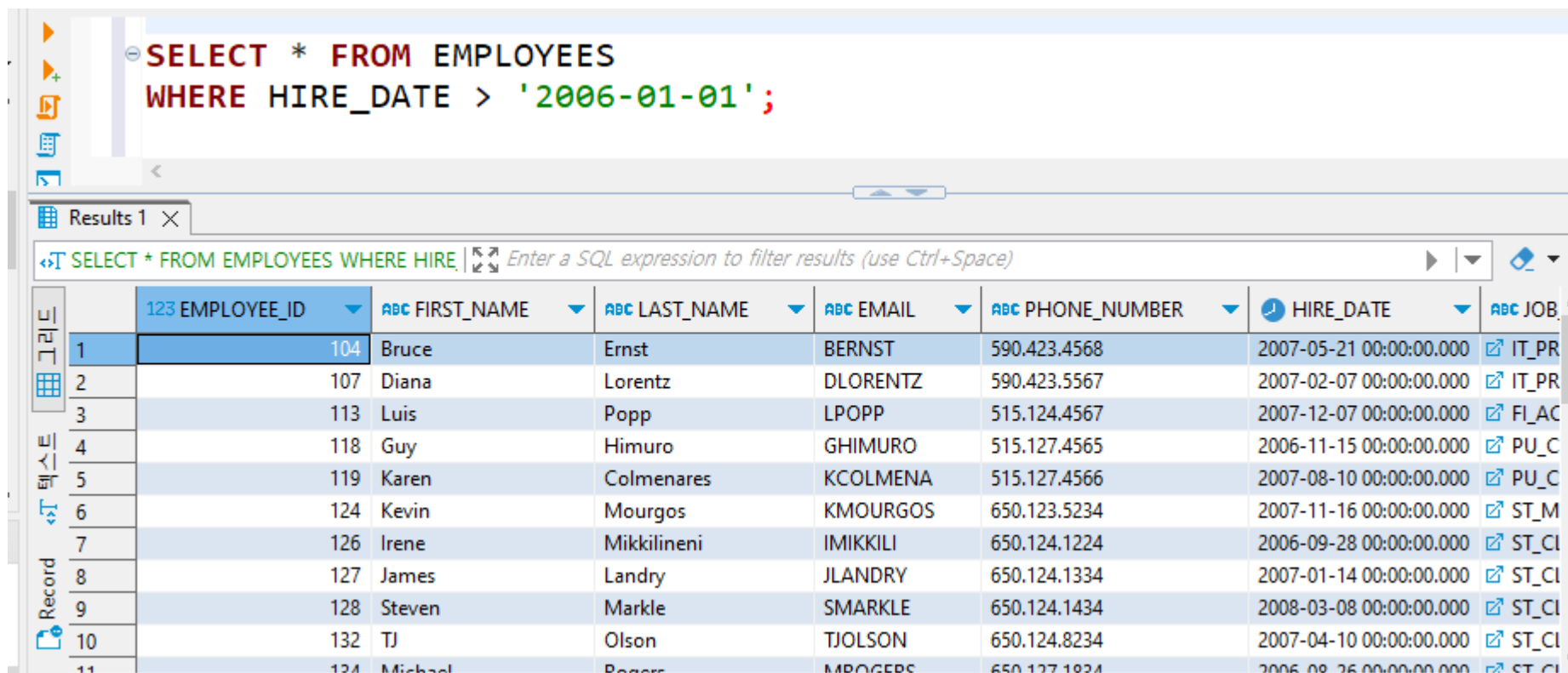


The screenshot shows a SQL IDE interface. At the top, a query is entered in the editor: `SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES WHERE FIRST_NAME != 'Diana';`. Below the editor, the results are displayed in a table. The table has 5 columns: `EMPLOYEE_ID`, `FIRST_NAME`, `LAST_NAME`, and `SALARY`. The results show 15 rows of employee data, starting with Steven King (ID 100) and ending with Alexander Khoo (ID 115). The first name 'Diana' is not present in the results.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	100	Steven	King	24,000
2	101	Neena	Kochhar	17,000
3	102	Lex	De Haan	17,000
4	103	Alexander	Hunold	9,000
5	104	Bruce	Ernst	6,000
6	105	David	Austin	4,800
7	106	Valli	Pataballa	4,800
8	108	Nancy	Greenberg	12,008
9	109	Daniel	Faviet	9,000
10	110	John	Chen	8,200
11	111	Ismael	Sciarra	7,700
12	112	Jose Manuel	Urman	7,800
13	113	Luis	Popp	6,900
14	114	Den	Raphaely	11,000
15	115	Alexander	Khoo	3,100

- WHERE

입사일이 2006년 1월 1일 이후인 직원 조회



```
SELECT * FROM EMPLOYEES
WHERE HIRE_DATE > '2006-01-01';
```

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB
1	104	Bruce	Ernst	BERNST	590.423.4568	2007-05-21 00:00:00.000	IT_PR
2	107	Diana	Lorentz	DLORENTZ	590.423.5567	2007-02-07 00:00:00.000	IT_PR
3	113	Luis	Popp	LPOPP	515.124.4567	2007-12-07 00:00:00.000	FI_AC
4	118	Guy	Himuro	GHIMURO	515.127.4565	2006-11-15 00:00:00.000	PU_C
5	119	Karen	Colmenares	KCOLMENEA	515.127.4566	2007-08-10 00:00:00.000	PU_C
6	124	Kevin	Mourgos	KMOURGOS	650.123.5234	2007-11-16 00:00:00.000	ST_M
7	126	Irene	Mikkilineni	IMIKKILI	650.124.1224	2006-09-28 00:00:00.000	ST_CI
8	127	James	Landry	JLANDRY	650.124.1334	2007-01-14 00:00:00.000	ST_CI
9	128	Steven	Markle	SMARKLE	650.124.1434	2008-03-08 00:00:00.000	ST_CI
10	132	TJ	Olson	TJOLSON	650.124.8234	2007-04-10 00:00:00.000	ST_CI

• Q3. 실습 (EMPLOYEES 테이블)

1. 직원 테이블에서 부서 번호가 90번인 직원들을 조회
→ 부서번호 컬럼명 : DEPARTMENT_ID
→ 조회할 컬럼 : FIRST_NAME, LAST_NAME, EMAIL
2. 월급이 10,000 달러 이상인 직원들을 조회
→ 조회할 컬럼 : JOB_ID, MANAGER_ID, DEPARTMENT_ID
3. 보너스를 포함한 연봉이 20,000 달러 이상인 직원들 조회
→ 조회할 컬럼 : EMPLOYEE_ID, FIRST_NAME, 연봉, HIRE_DATE

- 논리연산(AND, OR)

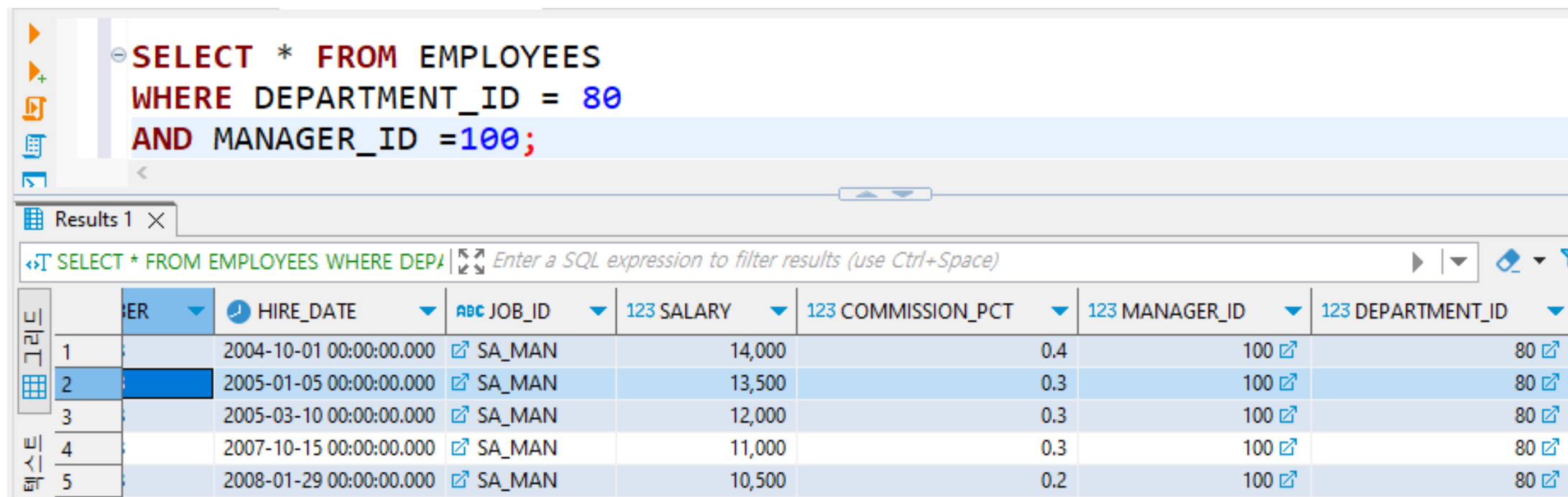
자바와 논리연산은 동일함.

AND 연산 : 두 피 연산자가 모두 참일 때 (자바의 &&)

OR 연산 : 두 피 연산자 중 하나 이상이 참일 때 (자바의 ||)

- 논리연산(AND)

부서 번호가 80번 이면서, 매니저 아이디가 100번인 직원 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

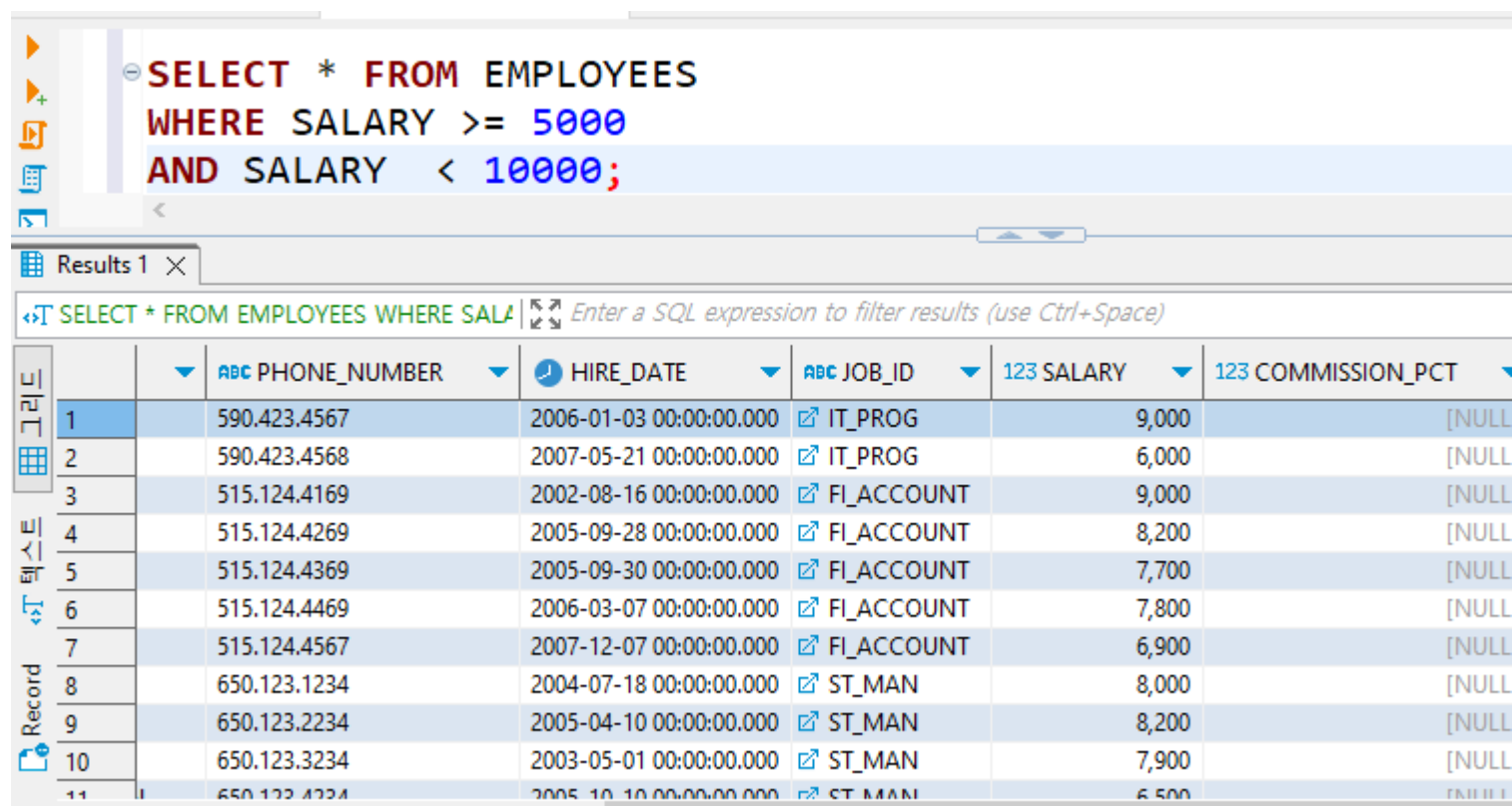
```
SELECT * FROM EMPLOYEES  
WHERE DEPARTMENT_ID = 80  
AND MANAGER_ID = 100;
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has 8 columns: EMPLOYEE_ID, LAST_NAME, HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, MANAGER_ID, and DEPARTMENT_ID. The results show 5 rows of data, all of which have a DEPARTMENT_ID of 80 and a MANAGER_ID of 100.

EMPLOYEE_ID	LAST_NAME	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	SCOTT	2004-10-01 00:00:00.000	SA_MAN	14,000	0.4	100	80
2	DEER	2005-01-05 00:00:00.000	SA_MAN	13,500	0.3	100	80
3	DEER	2005-03-10 00:00:00.000	SA_MAN	12,000	0.3	100	80
4	DEER	2007-10-15 00:00:00.000	SA_MAN	11,000	0.3	100	80
5	DEER	2008-01-29 00:00:00.000	SA_MAN	10,500	0.2	100	80

- 논리연산(AND)

월급이 5000달러 이상이면서, 1000달러 미만인 직원 조회



SELECT * FROM EMPLOYEES
WHERE SALARY >= 5000
AND SALARY < 10000;

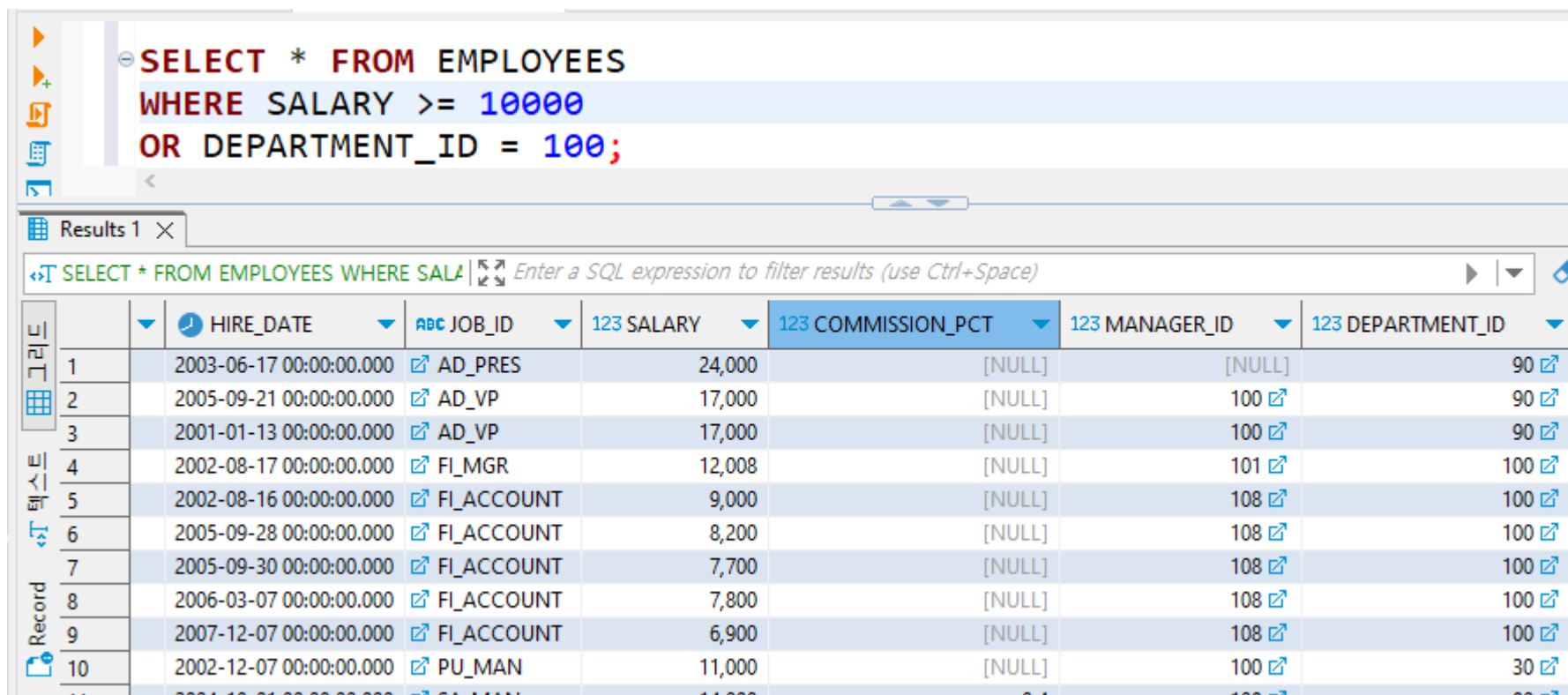
Results 1 X

SELECT * FROM EMPLOYEES WHERE SALARY >= 5000 AND SALARY < 10000; Enter a SQL expression to filter results (use Ctrl+Space)

	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT
1	590.423.4567	2006-01-03 00:00:00.000	IT_PROG	9,000	[NULL]
2	590.423.4568	2007-05-21 00:00:00.000	IT_PROG	6,000	[NULL]
3	515.124.4169	2002-08-16 00:00:00.000	FI_ACCOUNT	9,000	[NULL]
4	515.124.4269	2005-09-28 00:00:00.000	FI_ACCOUNT	8,200	[NULL]
5	515.124.4369	2005-09-30 00:00:00.000	FI_ACCOUNT	7,700	[NULL]
6	515.124.4469	2006-03-07 00:00:00.000	FI_ACCOUNT	7,800	[NULL]
7	515.124.4567	2007-12-07 00:00:00.000	FI_ACCOUNT	6,900	[NULL]
8	650.123.1234	2004-07-18 00:00:00.000	ST_MAN	8,000	[NULL]
9	650.123.2234	2005-04-10 00:00:00.000	ST_MAN	8,200	[NULL]
10	650.123.3234	2003-05-01 00:00:00.000	ST_MAN	7,900	[NULL]

- 논리연산(OR)

월급이 1000달러 이상 이거나, 부서 번호가 100번인 직원 조회



```
SELECT * FROM EMPLOYEES
WHERE SALARY >= 10000
OR DEPARTMENT_ID = 100;
```

Results 1

SELECT * FROM EMPLOYEES WHERE SALARY >= 10000 OR DEPARTMENT_ID = 100

	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	2003-06-17 00:00:00.000	AD_PRES	24,000	[NULL]	[NULL]	90
2	2005-09-21 00:00:00.000	AD_VP	17,000	[NULL]	100	90
3	2001-01-13 00:00:00.000	AD_VP	17,000	[NULL]	100	90
4	2002-08-17 00:00:00.000	FI_MGR	12,008	[NULL]	101	100
5	2002-08-16 00:00:00.000	FI_ACCOUNT	9,000	[NULL]	108	100
6	2005-09-28 00:00:00.000	FI_ACCOUNT	8,200	[NULL]	108	100
7	2005-09-30 00:00:00.000	FI_ACCOUNT	7,700	[NULL]	108	100
8	2006-03-07 00:00:00.000	FI_ACCOUNT	7,800	[NULL]	108	100
9	2007-12-07 00:00:00.000	FI_ACCOUNT	6,900	[NULL]	108	100
10	2002-12-07 00:00:00.000	PU_MAN	11,000	[NULL]	100	30

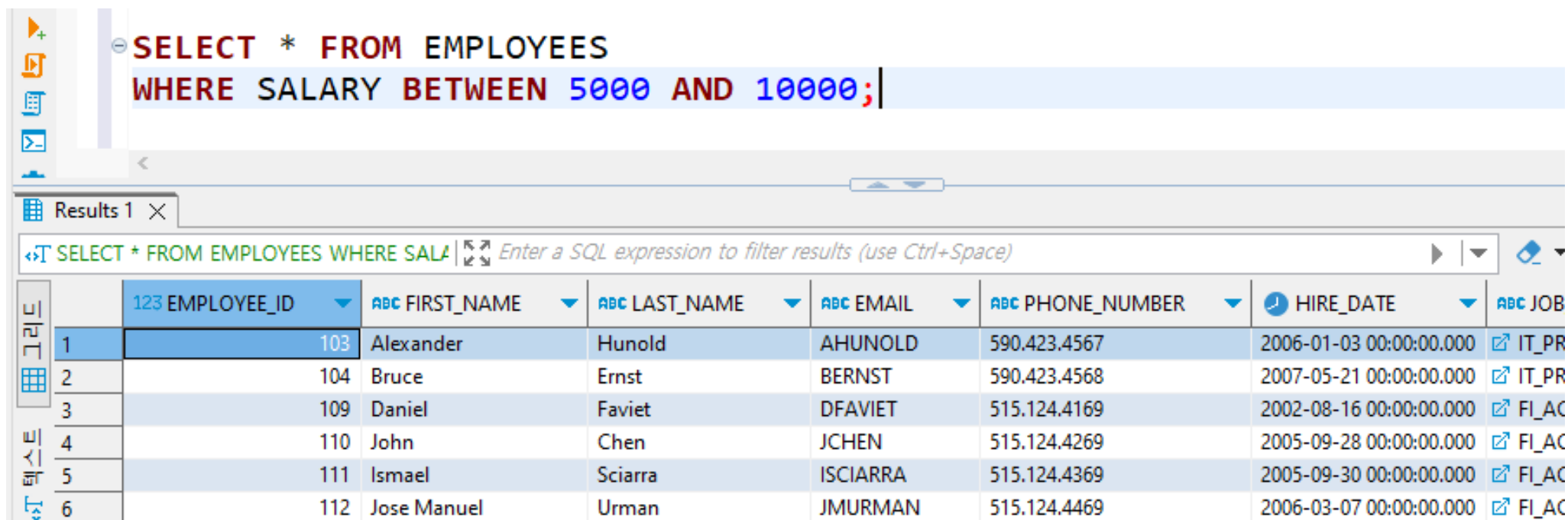
- **BETWEEN**

범위를 지정하여 값을 가져오고 싶을 때 사용

문법 : SELECT [컬럼명] FROM [테이블명] WHERE [컬럼명] BETWEEN [값] AND [값]

- BETWEEN

월급이 5,000달러 이상 10,000달러 미만인 직원 조회



The screenshot shows a SQL query editor with a query window and a results window. The query window contains the following SQL statement:

```
SELECT * FROM EMPLOYEES  
WHERE SALARY BETWEEN 5000 AND 10000;
```

The results window displays the results of the query, showing 6 rows of employee data. The columns are: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB. The first row is highlighted.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB
1	103	Alexander	Hunold	AHUNOLD	590.423.4567	2006-01-03 00:00:00.000	IT_PR
2	104	Bruce	Ernst	BERNST	590.423.4568	2007-05-21 00:00:00.000	IT_PR
3	109	Daniel	Faviet	DFAVIET	515.124.4169	2002-08-16 00:00:00.000	FI_AC
4	110	John	Chen	JCHEN	515.124.4269	2005-09-28 00:00:00.000	FI_AC
5	111	Ismael	Sciarra	ISCIARRA	515.124.4369	2005-09-30 00:00:00.000	FI_AC
6	112	Jose Manuel	Urman	JMURMAN	515.124.4469	2006-03-07 00:00:00.000	FI_AC

- BETWEEN

입사일이 2005년 2006년 사이인 직원 조회

```
SELECT * FROM EMPLOYEES  
WHERE HIRE_DATE BETWEEN '2005-01-01' AND '2006-01-01';
```

Results 1 X

'SELECT * FROM EMPLOYEES WHERE HIRE' Enter a SQL expression to filter results (use Ctrl+Space)

	123 EMPLOYEE_ID	ABC FIRST_NAME	ABC LAST_NAME	ABC EMAIL	ABC PHONE_NUMBER	HIRE_DATE
1	101	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21 00:00:00.000
2	105	David	Austin	DAUSTIN	590.423.4569	2005-06-25 00:00:00.000
3	110	John	Chen	JCHEN	515.124.4269	2005-09-28 00:00:00.000
4	111	Ismael	Sciarra	ISCIARRA	515.124.4369	2005-09-30 00:00:00.000
5	116	Shelli	Baida	SBAIDA	515.127.4563	2005-12-24 00:00:00.000
6	117	Sigal	Tobias	STOBIAS	515.127.4564	2005-07-24 00:00:00.000
7	121	Adam	Fripp	AFRIPP	650.123.2234	2005-04-10 00:00:00.000
8	123	Shanta	Vollman	SVOLLMAN	650.123.4234	2005-10-10 00:00:00.000

- LIKE

특정 패턴에 만족하는 지 확인해주는 구문
어떠한 문자들에 대한 패턴을 검색할 때 주로 사용됨

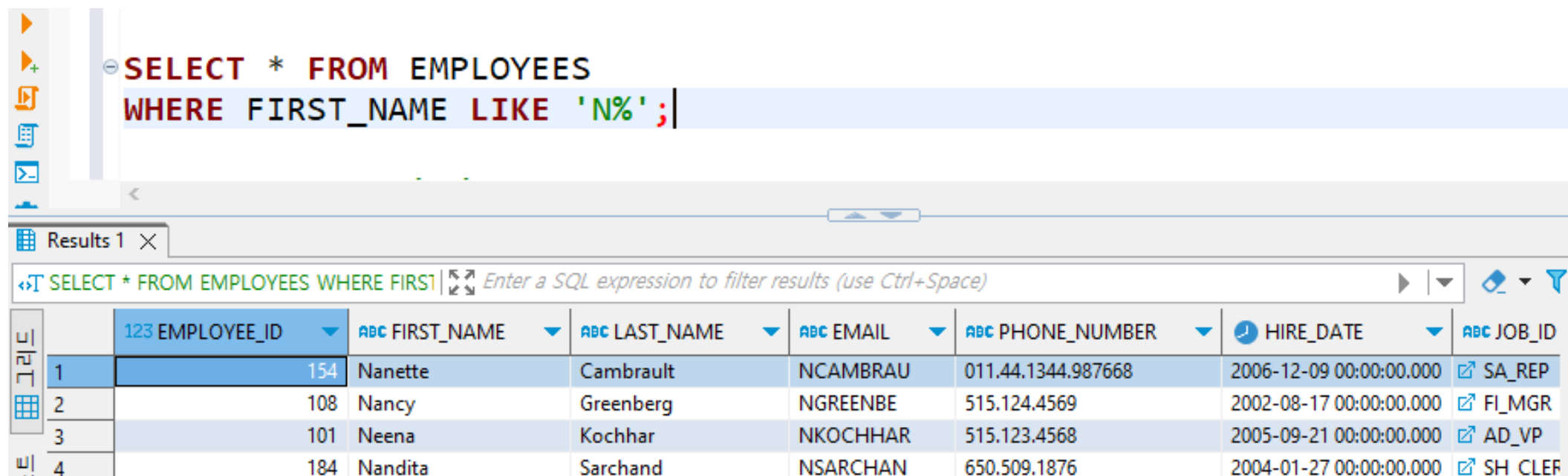
문법 : SELECT [컬럼명] FROM [테이블명] WHERE [컬럼명] LIKE [값]

표현식	설명
%	0글자 이상
_	1글자

- LIKE

FIRST_NAME 컬럼에서 "n"으로 시작하는 모든 문자열에 해당하는 데이터를 검색

% : 정규 표현식에서의 *를 생각하면 됨, 모든 문자열을 포함

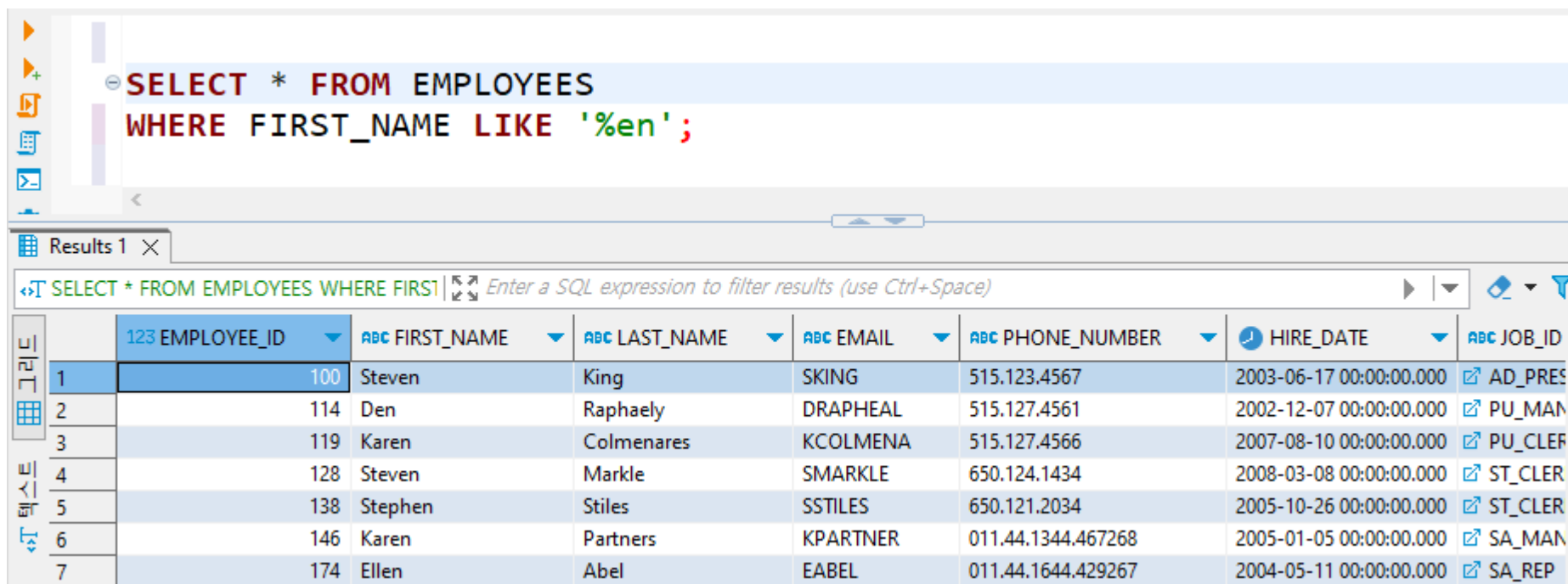


The screenshot shows a SQL IDE interface. At the top, a query is entered in a text area: `SELECT * FROM EMPLOYEES WHERE FIRST_NAME LIKE 'N%';`. Below the query, a tab labeled "Results 1" is active, displaying a table of query results. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The results show four rows of data, all with first names starting with 'N'.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	154	Nanette	Cambrault	NCAMBRAU	011.44.1344.987668	2006-12-09 00:00:00.000	SA_REP
2	108	Nancy	Greenberg	NGREENBE	515.124.4569	2002-08-17 00:00:00.000	FI_MGR
3	101	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21 00:00:00.000	AD_VP
4	184	Nandita	Sarchand	NSARCHAN	650.509.1876	2004-01-27 00:00:00.000	SH_CLERK

- LIKE

FIRST_NAME 컬럼에서 "en"으로 끝나는 모든 문자열에 해당하는 데이터를 검색



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

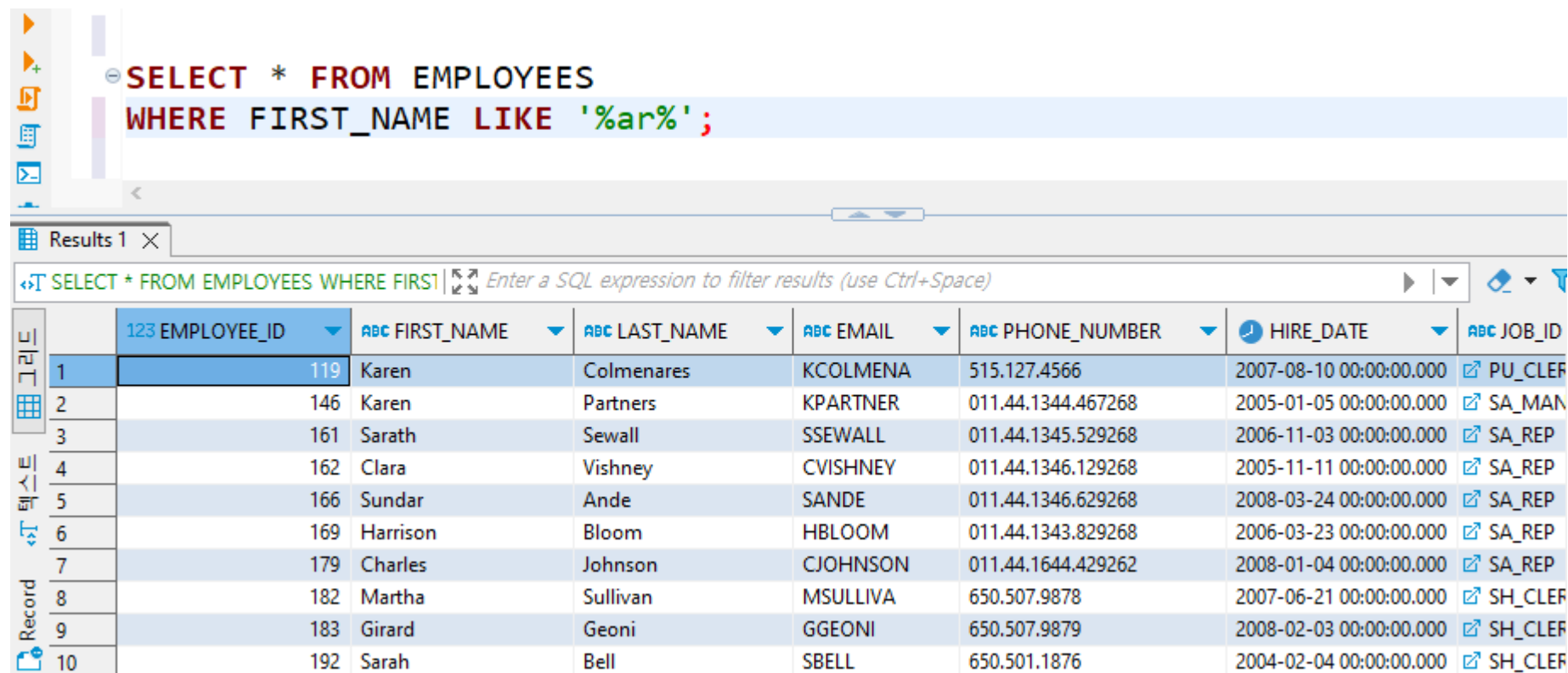
```
SELECT * FROM EMPLOYEES  
WHERE FIRST_NAME LIKE '%en';
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The results show 7 rows of data, all of which have first names ending in "en".

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	100	Steven	King	SKING	515.123.4567	2003-06-17 00:00:00.000	AD_PRES
2	114	Den	Raphaely	DRAPHEAL	515.127.4561	2002-12-07 00:00:00.000	PU_MAN
3	119	Karen	Colmenares	KCOLMENA	515.127.4566	2007-08-10 00:00:00.000	PU_CLEF
4	128	Steven	Markle	SMARKLE	650.124.1434	2008-03-08 00:00:00.000	ST_CLER
5	138	Stephen	Stiles	SSTILES	650.121.2034	2005-10-26 00:00:00.000	ST_CLER
6	146	Karen	Partners	KPARTNER	011.44.1344.467268	2005-01-05 00:00:00.000	SA_MAN
7	174	Ellen	Abel	EABEL	011.44.1644.429267	2004-05-11 00:00:00.000	SA_REP

- LIKE

FIRST_NAME 컬럼에서 문자열 중간에 "ar"이 해당하는 데이터를 검색



The screenshot shows a SQL IDE interface. At the top, a query editor contains the following SQL statement:

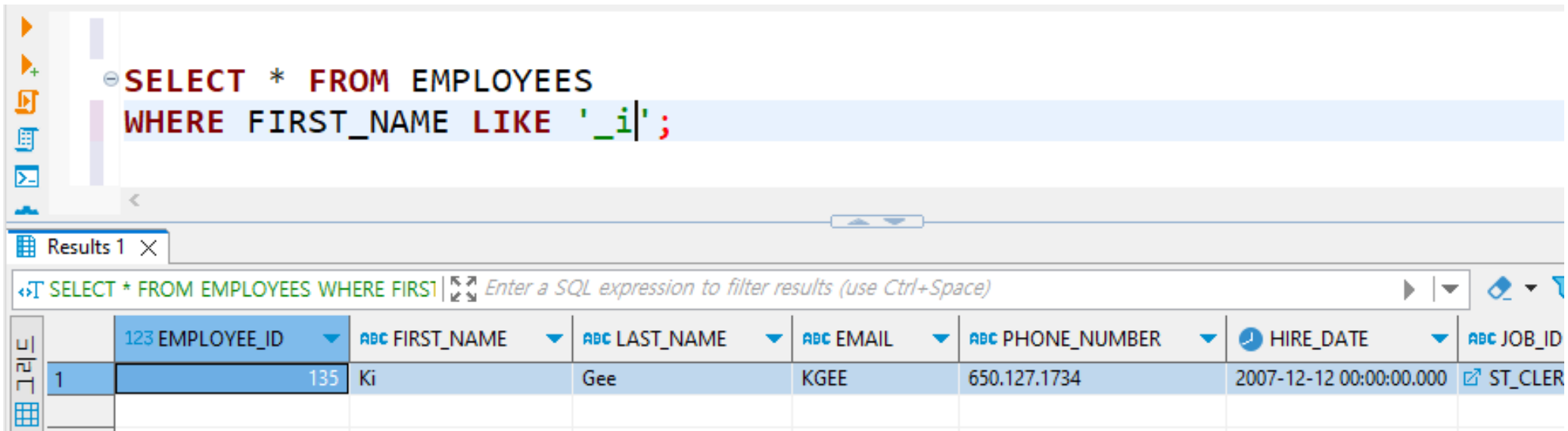
```
SELECT * FROM EMPLOYEES  
WHERE FIRST_NAME LIKE '%ar%';
```

Below the query editor, the results are displayed in a table. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The results are filtered to show only employees whose first name contains the substring 'ar'.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	119	Karen	Colmenares	KCOLMENA	515.127.4566	2007-08-10 00:00:00.000	PU_CLER
2	146	Karen	Partners	KPARTNER	011.44.1344.467268	2005-01-05 00:00:00.000	SA_MAN
3	161	Sarath	Sewall	SSEWALL	011.44.1345.529268	2006-11-03 00:00:00.000	SA_REP
4	162	Clara	Vishney	CVISHNEY	011.44.1346.129268	2005-11-11 00:00:00.000	SA_REP
5	166	Sundar	Ande	SANDE	011.44.1346.629268	2008-03-24 00:00:00.000	SA_REP
6	169	Harrison	Bloom	HBLOOM	011.44.1343.829268	2006-03-23 00:00:00.000	SA_REP
7	179	Charles	Johnson	CJOHNSON	011.44.1644.429262	2008-01-04 00:00:00.000	SA_REP
8	182	Martha	Sullivan	MSULLIVA	650.507.9878	2007-06-21 00:00:00.000	SH_CLER
9	183	Girard	Geoni	GGEONI	650.507.9879	2008-02-03 00:00:00.000	SH_CLER
10	192	Sarah	Bell	SBELL	650.501.1876	2004-02-04 00:00:00.000	SH_CLER

- LIKE

FIRST_NAME 컬럼에서 "i" 앞에 한글자가 존재하는 데이터 조회



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

```
SELECT * FROM EMPLOYEES  
WHERE FIRST_NAME LIKE '_i';
```

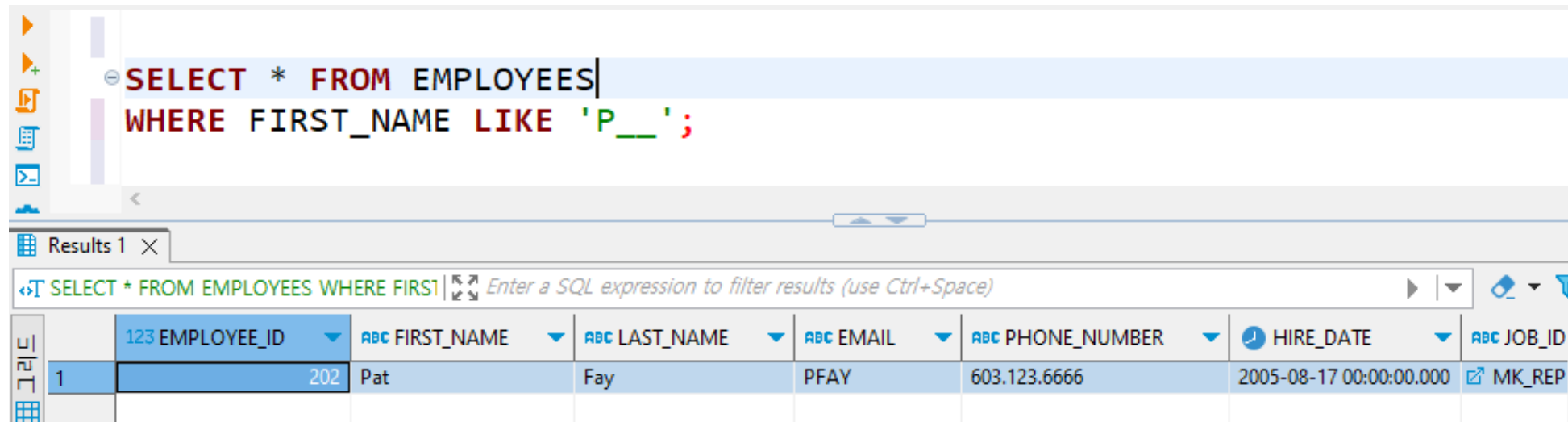
The bottom pane shows the results of the query in a table format. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The first row of data shows an employee with ID 135, first name 'Ki', last name 'Gee', email 'KGEE', phone number '650.127.1734', hire date '2007-12-12 00:00:00.000', and job ID 'ST_CLERK'.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	135	Ki	Gee	KGEE	650.127.1734	2007-12-12 00:00:00.000	ST_CLERK

• LIKE

FIRST_NAME 컬럼에서 "P"로 시작하고 뒤에 2글자가 존재하는 데이터 조회

* 언더바(_)를 두번 사용했으므로 2글자를 뜻함



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

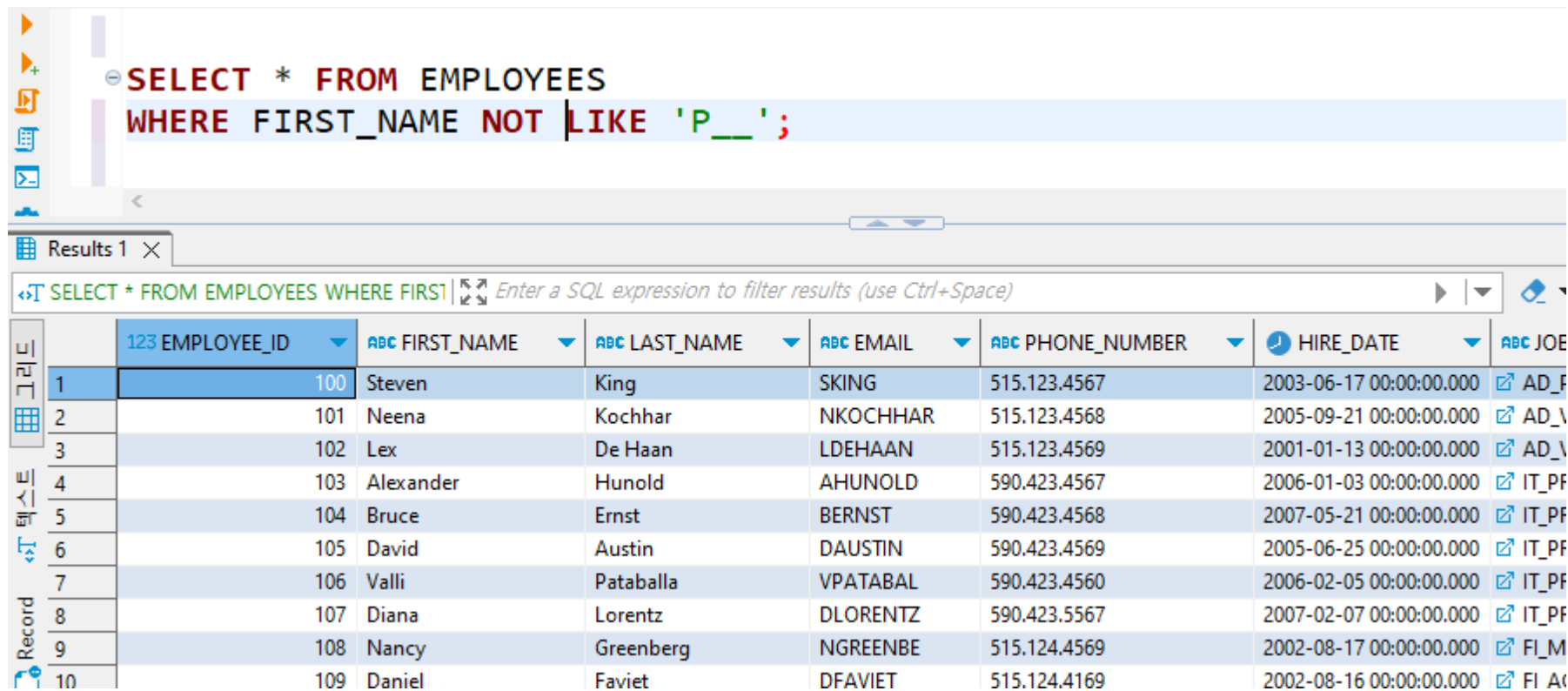
```
SELECT * FROM EMPLOYEES  
WHERE FIRST_NAME LIKE 'P__';
```

The bottom pane, titled "Results 1", shows the query results in a table format. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The first row of data is highlighted in blue.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	202	Pat	Fay	PFAY	603.123.6666	2005-08-17 00:00:00.000	MK_REP

- **NOT LIKE**

NOT LIKE를 사용하여 해당 조건에 만족하지 않는 데이터만을 조회할 수 있음
데이터가 **P로 시작하며 뒤에 2글자가 붙지 않는** 데이터 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

```
SELECT * FROM EMPLOYEES  
WHERE FIRST_NAME NOT LIKE 'P__';
```

The bottom pane shows the results of the query, titled "Results 1". The results are displayed in a table with 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB. The table contains 10 rows of data, with the first row highlighted in blue.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB
1	100	Steven	King	SKING	515.123.4567	2003-06-17 00:00:00.000	AD_F
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21 00:00:00.000	AD_V
3	102	Lex	De Haan	LDEHAAN	515.123.4569	2001-01-13 00:00:00.000	AD_V
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	2006-01-03 00:00:00.000	IT_PF
5	104	Bruce	Ernst	BERNST	590.423.4568	2007-05-21 00:00:00.000	IT_PF
6	105	David	Austin	DAUSTIN	590.423.4569	2005-06-25 00:00:00.000	IT_PF
7	106	Valli	Pataballa	VPATABAL	590.423.4560	2006-02-05 00:00:00.000	IT_PF
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	2007-02-07 00:00:00.000	IT_PF
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	2002-08-17 00:00:00.000	FI_M
10	109	Daniel	Faviet	DFAVIET	515.124.4169	2002-08-16 00:00:00.000	FI_A

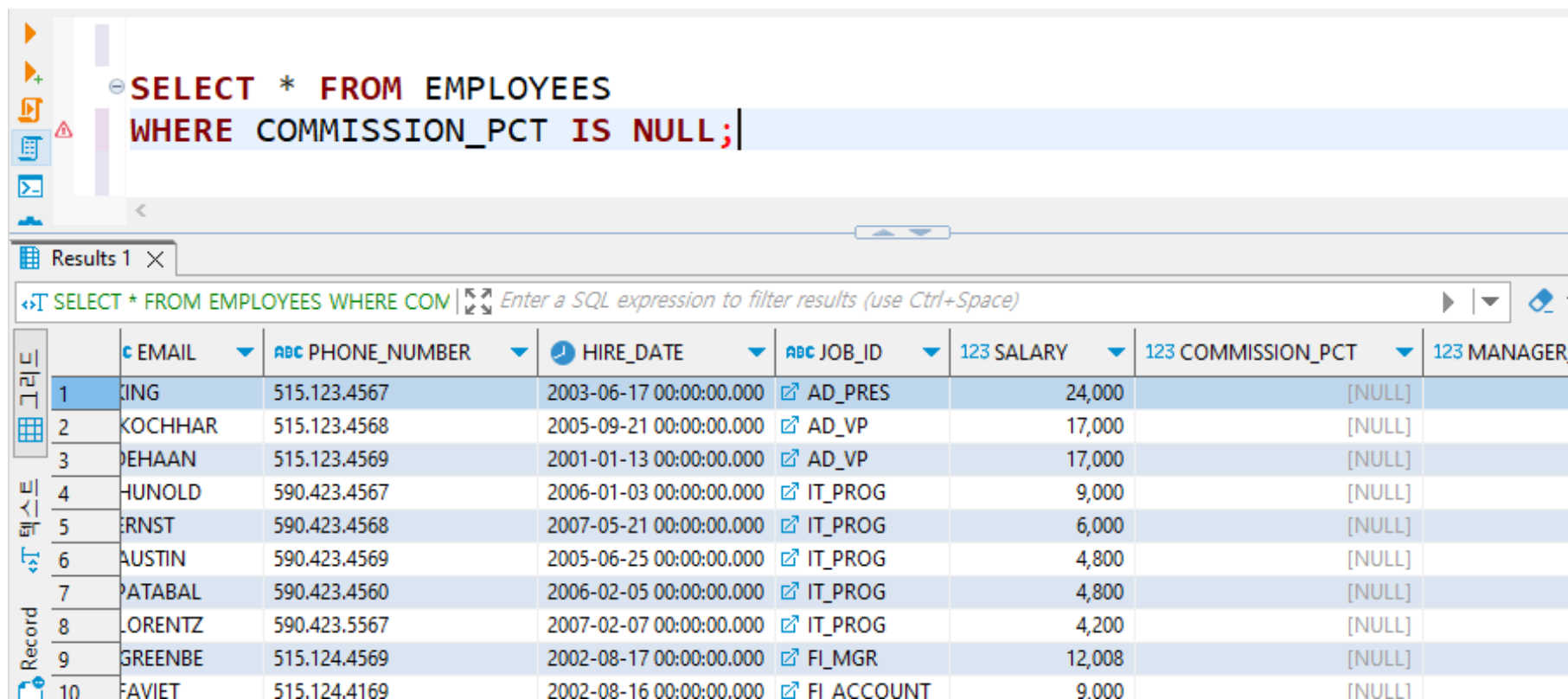
- IS NULL

컬럼에 있는 데이터의 값이 NULL 인지 비교하는 연산자

문법 : `SELECT [컬럼명] FROM [테이블명] WHERE [컬럼명] IS NULL`

- IS NULL

COMMISSION_PCT 컬럼의 값이 NULL인 데이터 조회

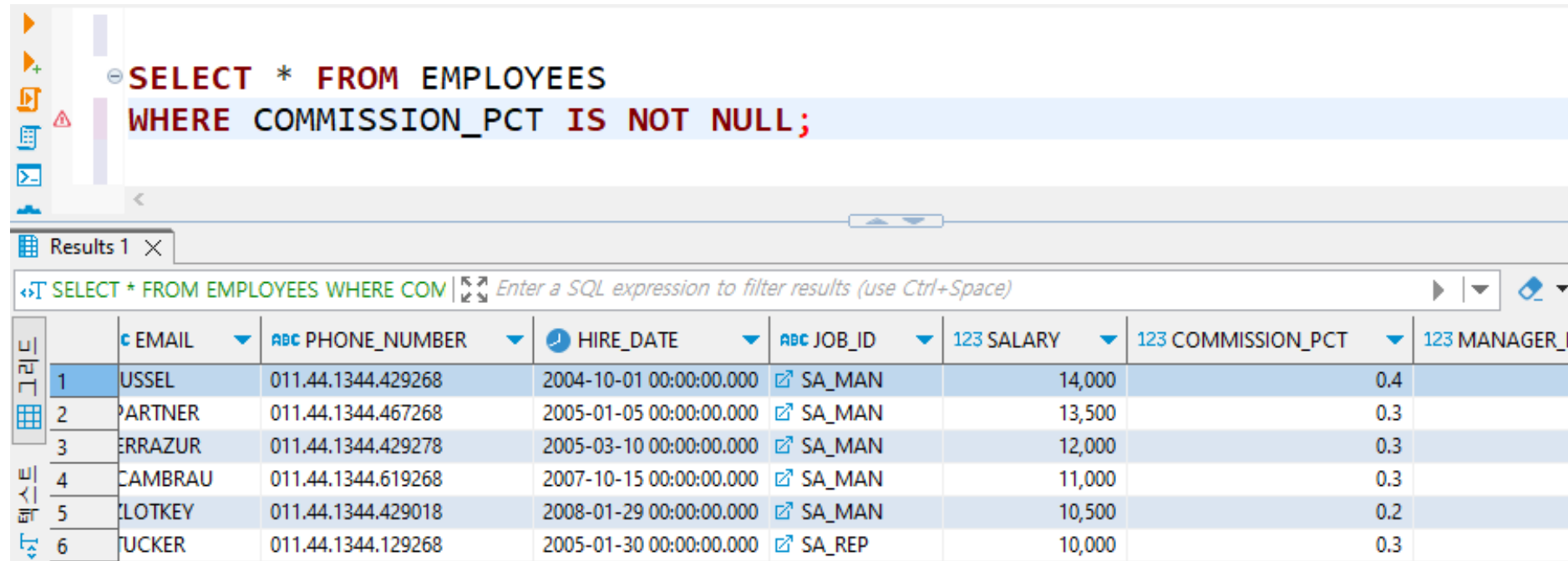


The screenshot shows a SQL IDE interface. The top pane contains the query: `SELECT * FROM EMPLOYEES WHERE COMMISSION_PCT IS NULL;`. The bottom pane, titled 'Results 1', displays the query results in a table. The table has 8 columns: EMAIL, PHONE_NUMBER, HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, and MANAGER_ID. The results show 10 rows of data, all of which have a NULL value in the COMMISSION_PCT column.

	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	ING	515.123.4567	2003-06-17 00:00:00.000	AD_PRES	24,000	[NULL]	
2	KOCHHAR	515.123.4568	2005-09-21 00:00:00.000	AD_VP	17,000	[NULL]	
3	DEHAAN	515.123.4569	2001-01-13 00:00:00.000	AD_VP	17,000	[NULL]	
4	HUNOLD	590.423.4567	2006-01-03 00:00:00.000	IT_PROG	9,000	[NULL]	
5	ERNST	590.423.4568	2007-05-21 00:00:00.000	IT_PROG	6,000	[NULL]	
6	AUSTIN	590.423.4569	2005-06-25 00:00:00.000	IT_PROG	4,800	[NULL]	
7	PATABAL	590.423.4560	2006-02-05 00:00:00.000	IT_PROG	4,800	[NULL]	
8	LORENTZ	590.423.5567	2007-02-07 00:00:00.000	IT_PROG	4,200	[NULL]	
9	GREENBE	515.124.4569	2002-08-17 00:00:00.000	FI_MGR	12,008	[NULL]	
10	FAVIET	515.124.4169	2002-08-16 00:00:00.000	FI_ACCOUNT	9,000	[NULL]	

- IS NOT NULL

COMMISSION_PCT 컬럼의 값이 NULL이 아닌 데이터 조회



The screenshot shows a SQL IDE interface. At the top, a query is entered in the editor: `SELECT * FROM EMPLOYEES WHERE COMMISSION_PCT IS NOT NULL;`. Below the editor, the 'Results 1' tab is active, displaying a table of query results. The table has 8 columns: EMAIL, PHONE_NUMBER, HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, and MANAGER_ID. There are 6 rows of data. The first row is highlighted in blue. The table is titled 'Results 1' and has a search bar above it with the text 'Enter a SQL expression to filter results (use Ctrl+Space)'.

	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	USSEL	011.44.1344.429268	2004-10-01 00:00:00.000	SA_MAN	14,000	0.4	
2	PARTNER	011.44.1344.467268	2005-01-05 00:00:00.000	SA_MAN	13,500	0.3	
3	ERRAZUR	011.44.1344.429278	2005-03-10 00:00:00.000	SA_MAN	12,000	0.3	
4	CAMBRAU	011.44.1344.619268	2007-10-15 00:00:00.000	SA_MAN	11,000	0.3	
5	LOTKEY	011.44.1344.429018	2008-01-29 00:00:00.000	SA_MAN	10,500	0.2	
6	TUCKER	011.44.1344.129268	2005-01-30 00:00:00.000	SA_REP	10,000	0.3	

- IN

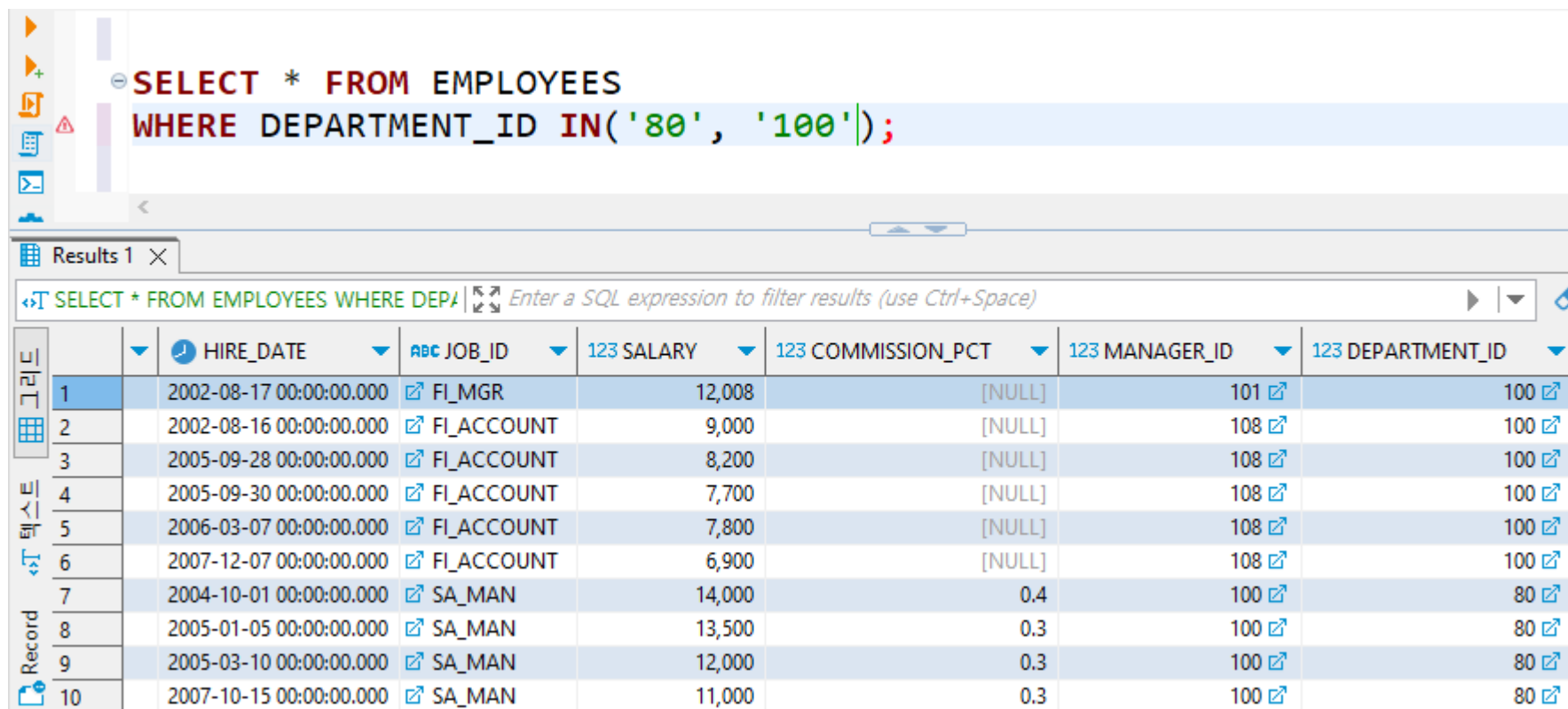
여러 개의 값을 제시하고 해당하는 값에 일치하는 데이터가 있는지 조회

문법 : `SELECT [컬럼명] FROM [테이블명] WHERE [컬럼명] IN('값1', '값2', '값3');`

* AND와 OR는 한가지의 값을 가지고 비교를 한다면, IN은 여러 개의 값을 한번에 비교할 수 있는 연산자

• IN

DEPARTMENT_ID의 값이 80 또는 100인 데이터를 조회

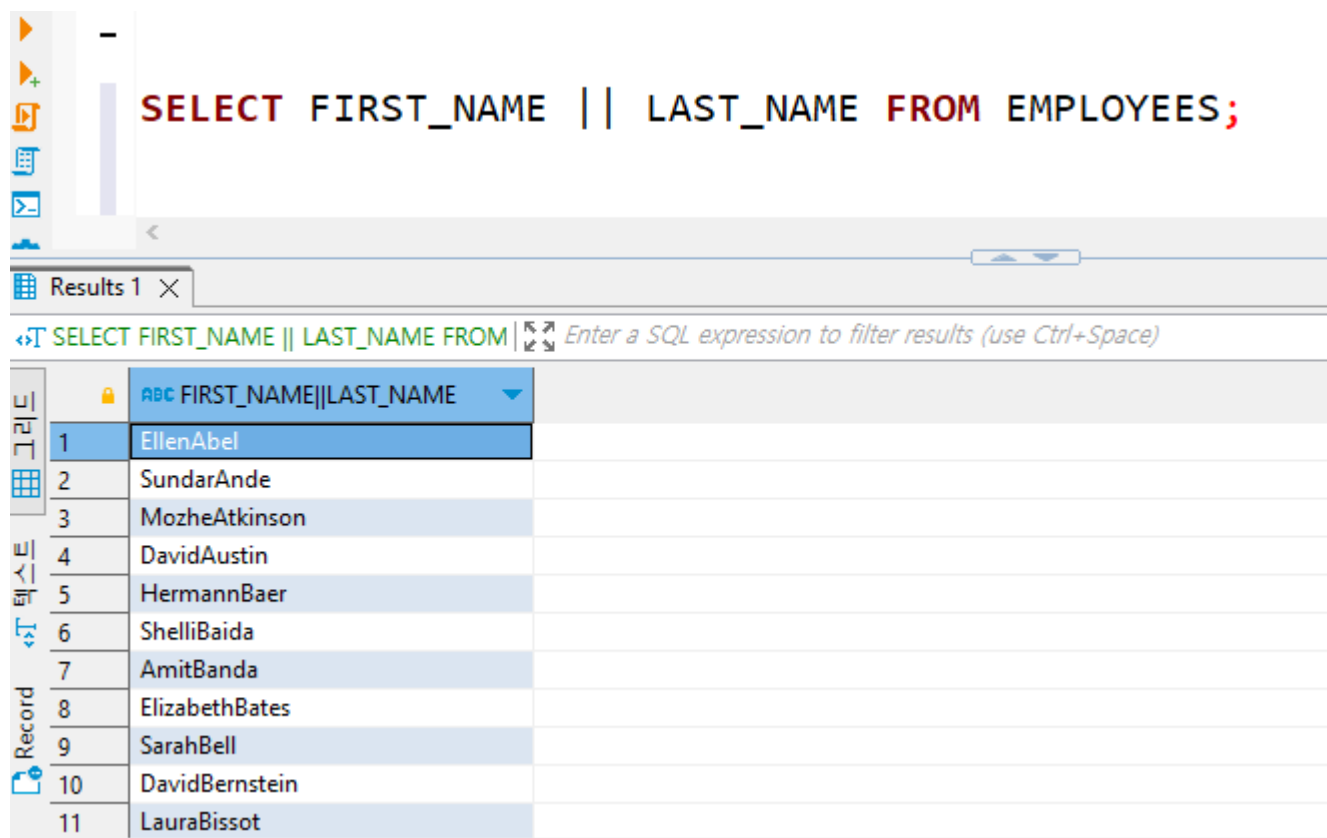


The screenshot shows a SQL IDE interface. At the top, a query is entered in a text area: `SELECT * FROM EMPLOYEES WHERE DEPARTMENT_ID IN('80', '100');`. Below the query, a tab labeled "Results 1" is active, displaying a table of results. The table has 8 columns: HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, MANAGER_ID, and DEPARTMENT_ID. The results are sorted by DEPARTMENT_ID, with rows for DEPARTMENT_ID 100 appearing first, followed by rows for DEPARTMENT_ID 80. The table contains 10 rows of data.

	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	2002-08-17 00:00:00.000	FI_MGR	12,008	[NULL]	101	100
2	2002-08-16 00:00:00.000	FI_ACCOUNT	9,000	[NULL]	108	100
3	2005-09-28 00:00:00.000	FI_ACCOUNT	8,200	[NULL]	108	100
4	2005-09-30 00:00:00.000	FI_ACCOUNT	7,700	[NULL]	108	100
5	2006-03-07 00:00:00.000	FI_ACCOUNT	7,800	[NULL]	108	100
6	2007-12-07 00:00:00.000	FI_ACCOUNT	6,900	[NULL]	108	100
7	2004-10-01 00:00:00.000	SA_MAN	14,000	0.4	100	80
8	2005-01-05 00:00:00.000	SA_MAN	13,500	0.3	100	80
9	2005-03-10 00:00:00.000	SA_MAN	12,000	0.3	100	80
10	2007-10-15 00:00:00.000	SA_MAN	11,000	0.3	100	80

• 연결 연산자

여러 개의 컬럼 값을 하나의 컬럼처럼 연결 시켜줄 수 있는 연산자



The screenshot shows a SQL IDE interface. At the top, a query is entered in the editor: `SELECT FIRST_NAME || LAST_NAME FROM EMPLOYEES;`. Below the editor, the 'Results 1' tab is active, displaying the output of the query. The results are shown in a table with 11 rows. The first row is highlighted in blue. The table has two columns: 'FIRST_NAME' and 'LAST_NAME', which are concatenated into a single column labeled 'ABC FIRST_NAME||LAST_NAME'.

	ABC FIRST_NAME LAST_NAME
1	EllenAbel
2	SundarAnde
3	MozheAtkinson
4	DavidAustin
5	HermannBaer
6	ShelliBaida
7	AmitBanda
8	ElizabethBates
9	SarahBell
10	DavidBernstein
11	LauraBissot